

POLITECNICO
MILANO 1863

Reti Logiche

Panoramica sulle Codifiche

Rappresentazione dell'informazione numerica

- Un numero intero è rappresentato da un *vettore di bit*

$$B = b_{n-1}\ldots b_1 b_0 \qquad b_i = \{0, 1\}$$

- Un vettore di *n* bit consente di rappresentare i numeri naturali nell'intervallo da 0 a 2^n-1.
- Il valore di *B* è dato da: Il valore

$$V(B) = b_{n-1}\times 2^{n-1} + \ldots + b_1\times 2^1 + b_0\times 2^0$$

- **Esempio:** 1 0 1 1 0

$$V(B) = 1x2^4 + 0x2^3 + 1x2^2 + 1x2^1 + 0x2^0 =$$

$$= 16 + 0 + 4 + 2 + 0 = 22$$

Rappresentazione di numeri relativi (1)

Modulo e segno:

- Il bit più significativo è il segno: 0 numero positivo, 1 numero negativo

$$V(B) = (-1)^{b_{n-1}} \times (b_{n-2} \times 2^{n-2} + \ldots + b_1 \times 2^1 + b_0 \times 2^0)$$

- Per ottenere l'opposto di N è sufficiente cambiare il bit di segno
- Finestra valori rappresentabili simmetrica, $[-2^{n-1}-1; +2^{n-1}-1]$
- Doppia rappresentazione dello 0

Esempio: 1 0 1 1 0

$$V(B) = (-1)^1 \times (0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0) =$$

$$= -1 (0 + 4 + 2 + 0) = -6$$

Rappresentazione di numeri relativi (2)

Complemento a 2

- Valore e segno formano una unica codifica

$$V(B) = -b_{n-1} \times 2^{n-1} + \ldots + b_1 \times 2^1 + b_0 \times 2^0$$

- Finestra valori rappresentabili asimmetrica $[-2^{n-1}-1; +2^{n-1}]$
- Singola rappresentazione dello 0
- Opposto di N: complemento bit a bit e si somma 000…01

Esempio: 1 0 1 1 0

$$V(B) = -1x2^4 + 0x2^3 + 1x2^2 + 1x2^1 + 0x2^0 =$$

$$= -16 + 0 + 4 + 2 + 0 = -10$$

Aritmetica binaria

- I sistemi digitali adottano il sistema binario per rappresentare i numeri
- Essi eseguono operazioni aritmetiche usando l'aritmetica binaria
- Nel corso vedremo algoritmi e architetture per eseguire le principali operazioni su numeri Naturali e numeri Relativi
  - Addizione
  - Sottrazione
  - Moltiplicazione
  - Divisione
- Diverso pattern di progettazione con attenzione alla regolarità, modularità e scalabilità dell'implementazione.



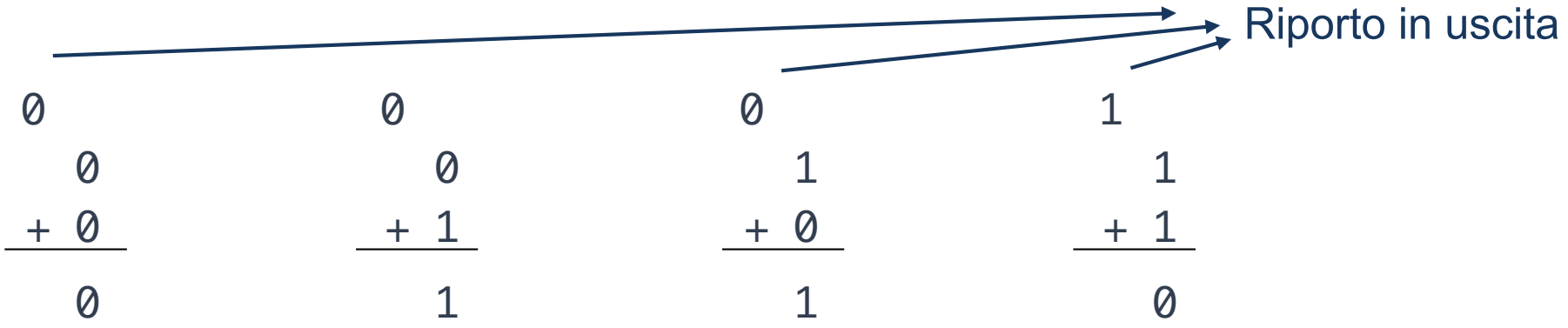
POLITECNICO
MILANO 1863

Reti Logiche

Addizionatori semplici per numeri naturali

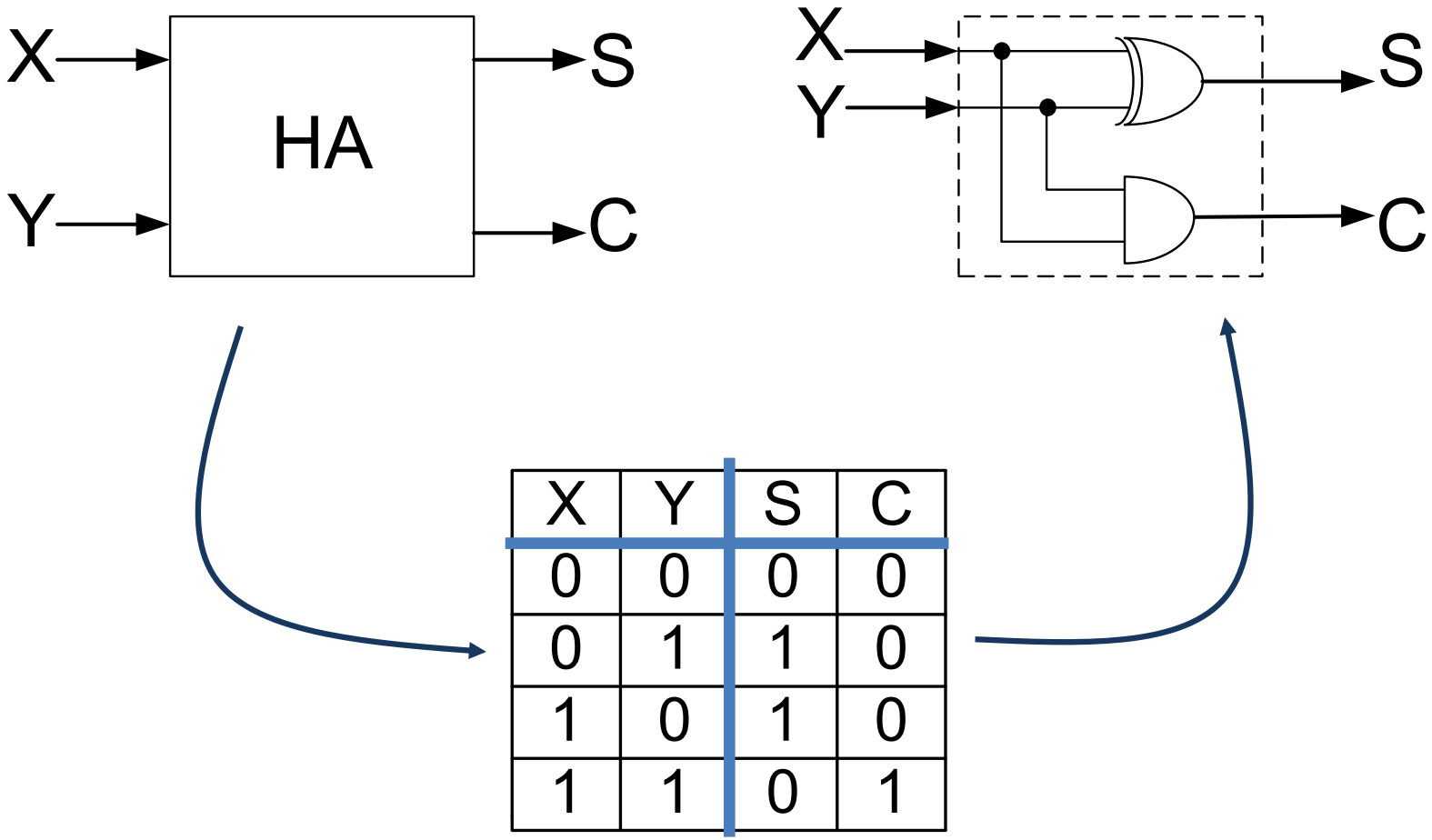
Addizione senza segno

- La somma di numeri positivi si esegue sommando coppie di bit parallele, partendo da destra.
- Si ha riporto, diverso da 0, quando si deve eseguire la somma 1+1
- Regole per la somma:



- Utilizzando queste regole in modo diretto è possibile
  - Realizzare **sommatori modulari**
  - Composti da blocchi elementari identici
  - Circuiti aritmetici di questo tipo sono detti **bit-slice**

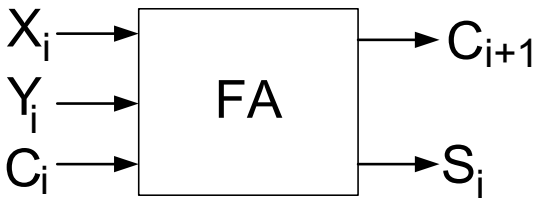
Half adder



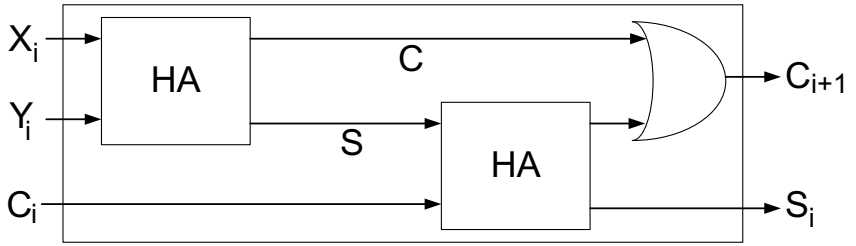
X	Y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Addizione senza segno: Full adder

- Un sommatore bit-slice è strutturato in modo che in posizione i-esima:
  - Riceve in ingresso i bit x_i e y_i degli operandi ed il riporto c_i del modulo precedente
  - Produce la somma s_i ed il riporto c_{i+1}
- Full Adder

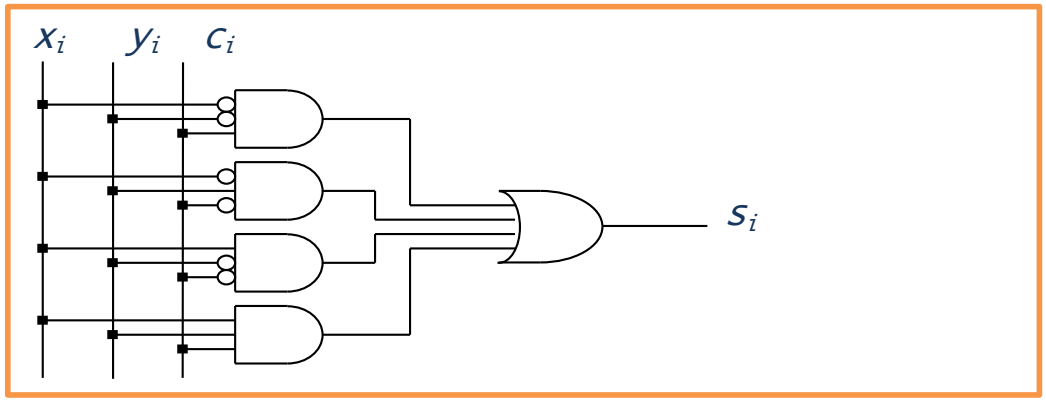


X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



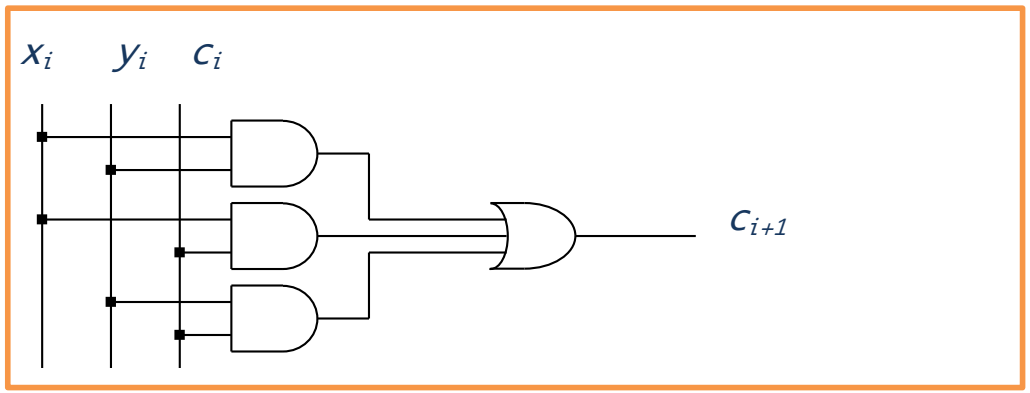
Full Adder – Circuiti Equivalenti

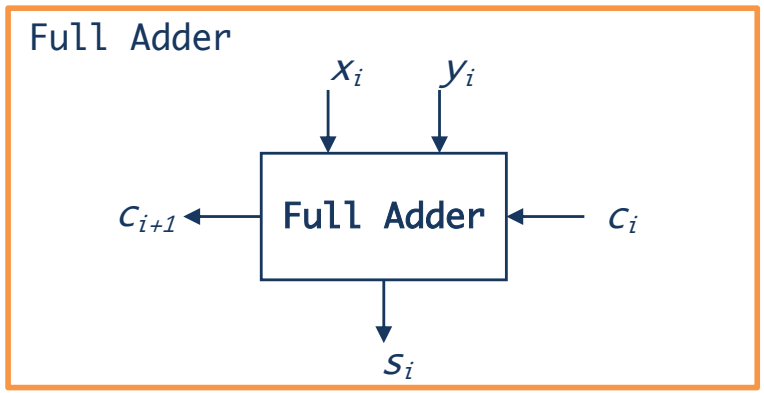
$$s_i = \overline{x}_i \overline{y}_i c_i + \overline{x}_i y_i \overline{c}_i + x_i \overline{y}_i \overline{c}_i + x_i y_i c_i$$



X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

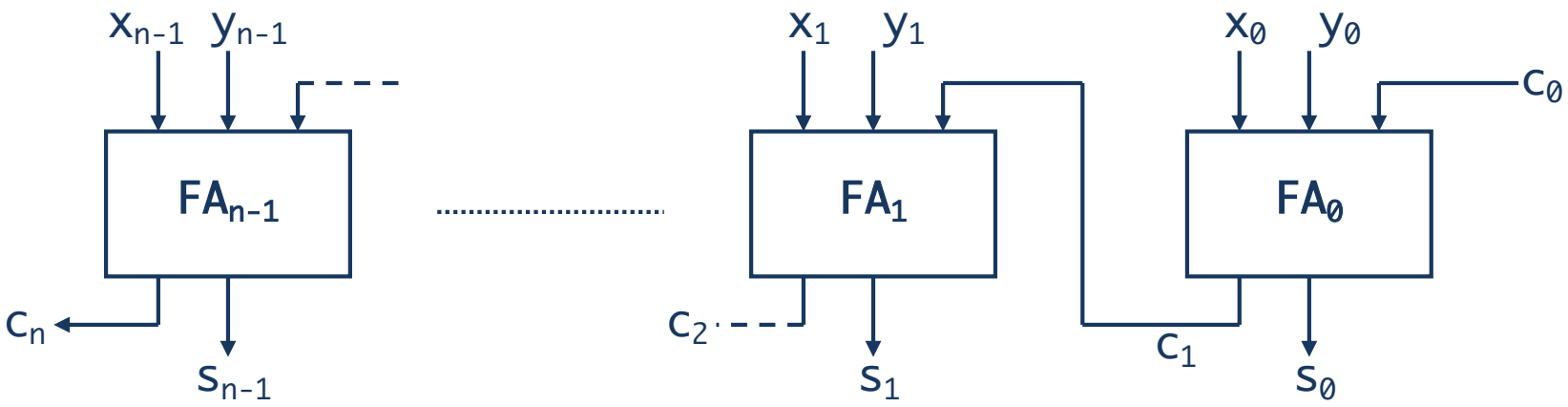
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$





Addizione senza segno - Ripple-Carry a n bit

- Architettura a propagazione di riporto (Ripple-Carry):



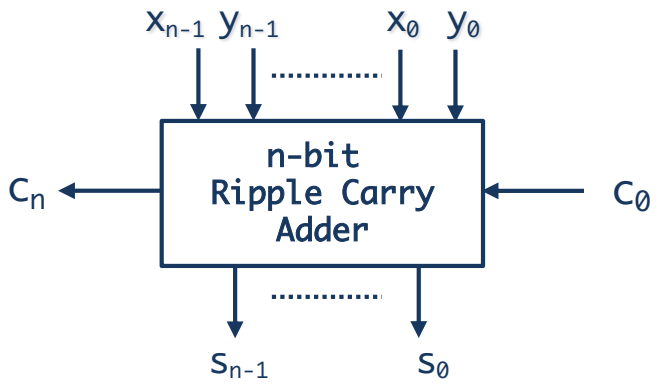
- Siano T_s e T_r i ritardi per il calcolo della somma e del riporto *i-esimi* rispettivamente
- **Prestazioni:** il ritardo totale per ottenere tutti i bit della somma è dato dall'espressione:

$$T_{tot} = (n-1)T_r + T_s$$

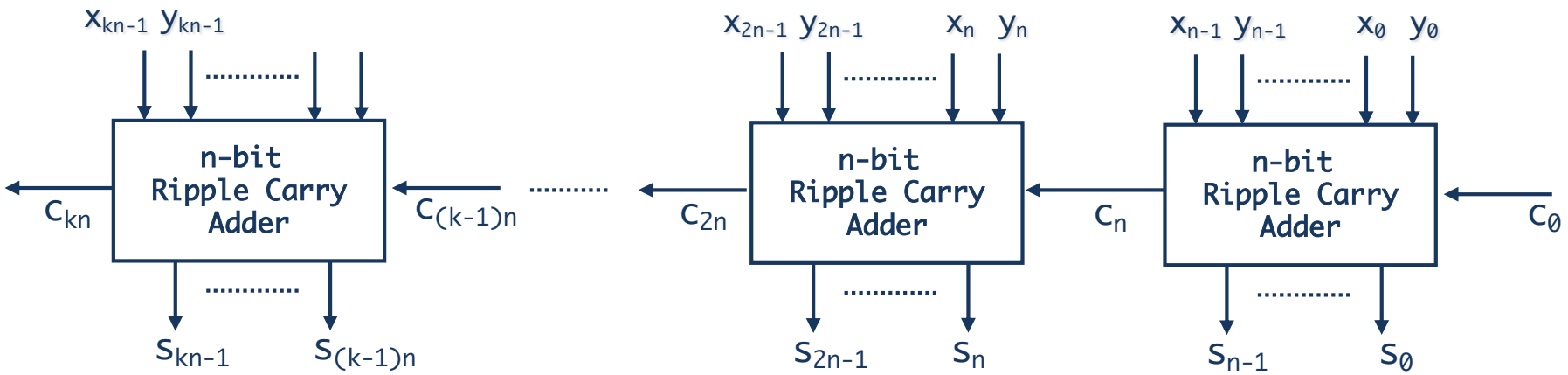
- Il **percorso critico** è quindi quello del **riporto**

Ripple Carry: architettura a blocchi

N-bit Ripple-Carry Adder



Ripple-Carry Block Architecture





POLITECNICO
MILANO 1863

Reti Logiche

Addizionatori veloci per numeri naturali

Addizione veloce e funzioni di generazione e di propagazione

- Necessità di ottenere un **sommatore** con **prestazioni migliori**
- Le espressioni di somma e riporto per lo stadio *i* sono:

$$s_i = x_i'y_i'c_i + x_i'y_ic_i' + x_iy_i'c_i' + x_iy_ic_i$$

$$c_{i+1} = x_iy_i + x_ic_i + y_ic_i = x_iy_i + (x_i + y_i)c_i$$

- L'espressione del riporto in uscita può essere riscritta come:

$$c_{i+1} = G_i + P_ic_i \quad \text{con} \quad G_i = x_iy_i \quad \text{e} \quad P_i = x_i + y_i$$

- Le funzioni G_i e P_i sono dette funzioni di generazione e propagazione
  - G_i: se $x_i=y_i=1$, allora il riporto in uscita **deve** essere generato
  - P_i: se x_i o $y_i=1$ e $c_i=1$, allora il riporto in ingresso **deve** essere propagato
  - Possono essere calcolate in parallelo, per tutti gli stadi, rispetto alle rispettive somme.

Calcolo dei riporti in parallelo

- L'espressione $c_{i+1} = G_i + P_i c_i$ può essere calcolata in modo **iterativo**.
- Sostituendo $c_i = G_{i-1} + P_{i-1} c_{i-1}$ nell'espressione di c_{i+1} si ha:

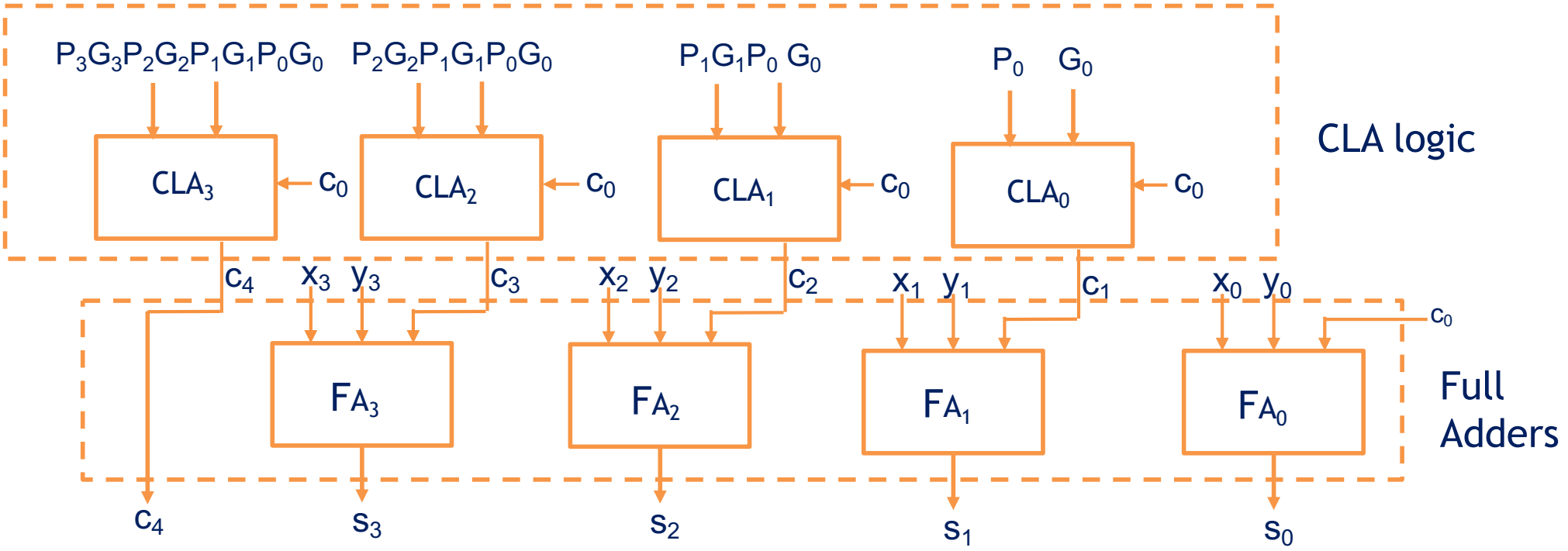
$$c_{i+1} = G_i + P_i(G_{i-1}+P_{i-1}c_{i-1}) = G_i + P_i G_{i-1} + P_i P_{i-1} c_{i-1}$$

- Continuando con l'**espansione** fino a c_0 si ottiene:

$$c_{i+1} = G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \ldots + P_i P_{i-1} \ldots P_1 G_0 + P_i P_{i-1} \ldots P_1 P_0 c_0$$

- I riporti in uscita di ogni singolo stadio possono essere calcolati tutti in parallelo e con ritardo identico (realizzazione SOP) tramite:
  - le funzioni di generazione e propagazione G_i e P_i
  - il riporto in ingresso allo stadio 0, c_0
- I sommatori che sfruttano il meccanismo della generazione dei riporti in anticipo sono detti Carry-Look-Ahead Adders o CLA Adders

Struttura di un CLA a 4 bit



$$c_1 = G_0 + P_0c_0$$

$$c_2 = G_1 + P_1G_0 + P_1P_0c_0$$

$$c_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0c_0$$

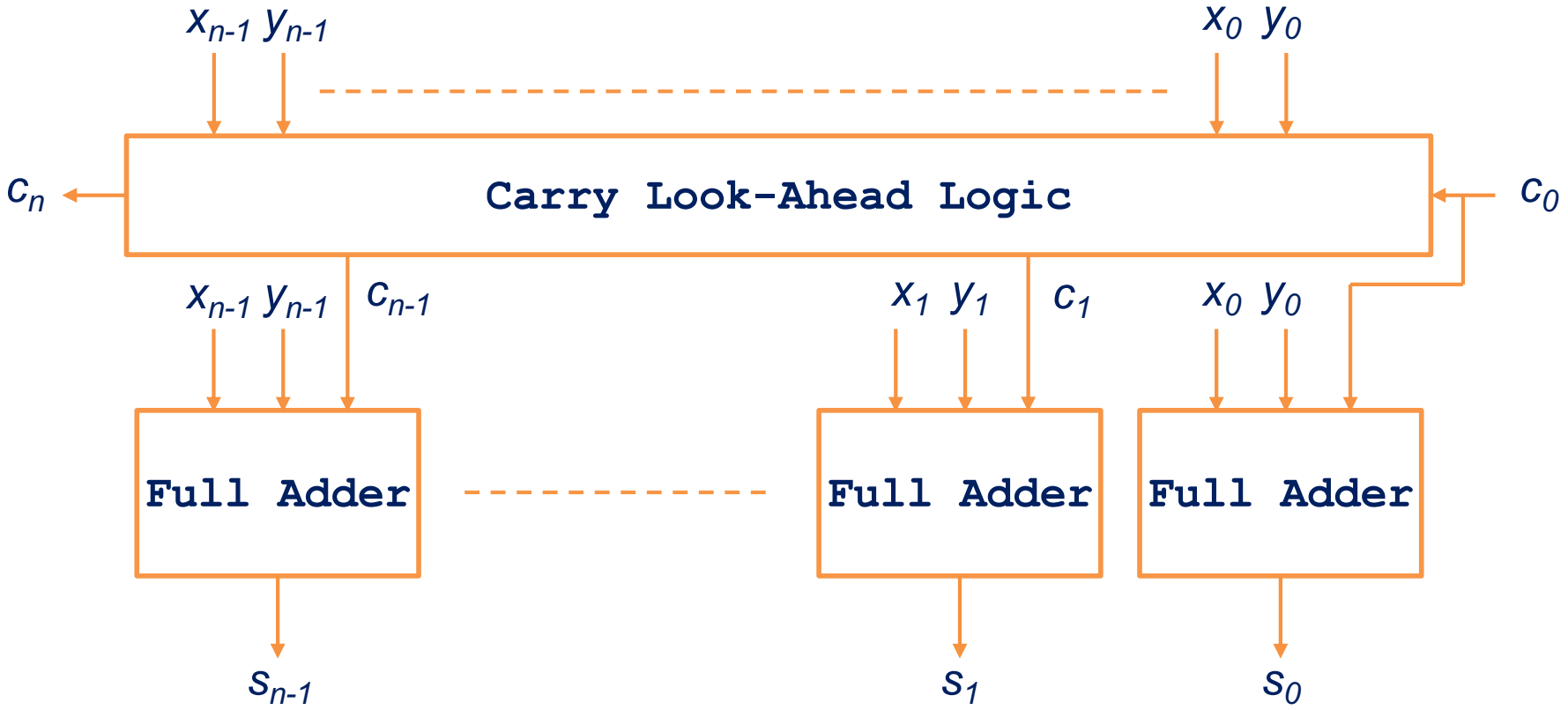
$$c_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0c_0$$

dove

$G_i = x_i y_i$

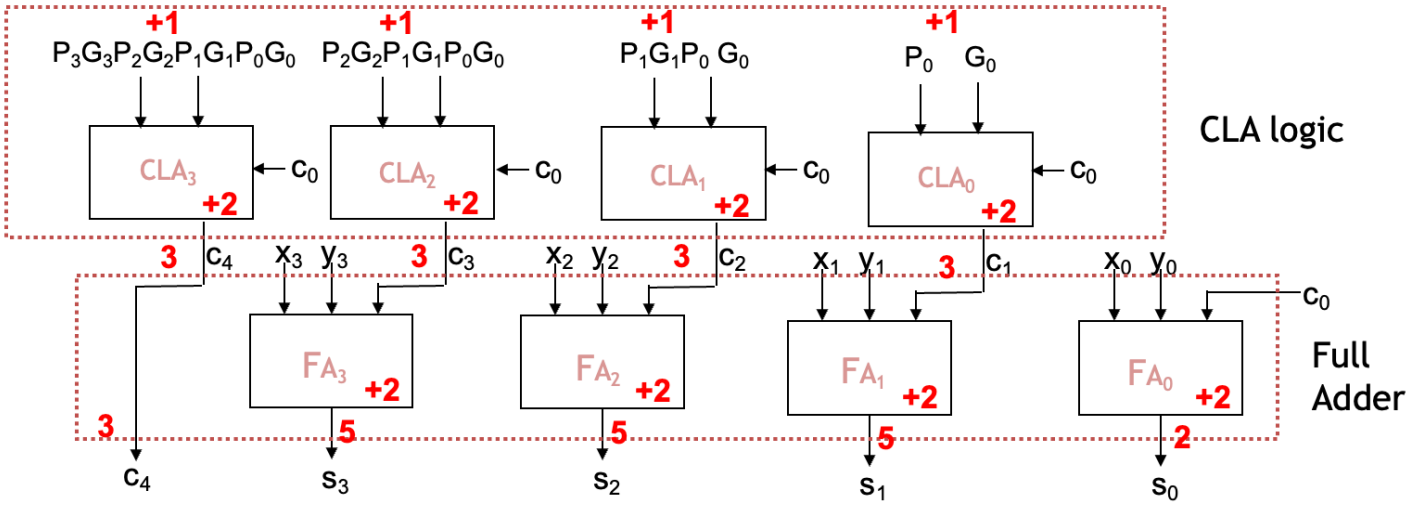
$P_i = x_i + y_i$

Sommatori Carry Look-Ahead



Addizione veloce: Calcolo delle prestazioni

- Il ritardo totale per ottenere tutte le somme ed il riporto più a sinistra c_{i+1} è dato dalla somma di:
  - Un ritardo di porta per il calcolo delle funzioni di generazione e di propagazione ($G_i = x_i y_i$ e $P_i = x_i + y_i$)
  - Due ritardi di porta logica per calcolare il riporto *i*-esimo (SOP)
  - Due ritardi di porta logica per calcolare la somma *i*-esima (SOP)
- Totale: 5 ritardi di porta logica



Addizione veloce: Calcolo delle prestazioni

- Il ritardo totale per ottenere tutte le somme ed il riporto più a sinistra c_{i+1} è dato dalla somma di:
  - Un ritardo di porta per il calcolo delle funzioni di generazione e di propagazione ($G_i = x_i y_i$ e $P_i = x_i + y_i$)
  - Due ritardi di porta logica per calcolare il riporto *i*-esimo (SOP)
  - Due ritardi di porta logica per calcolare la somma *i*-esima (SOP)
- Totale: 5 ritardi di porta logica

- **Il ritardo è costante e indipendente dalla lunghezza degli operandi**

- Problema:
  - **La realizzazione circuitale** dei moduli che calcolano i riporti per **operandi lunghi** (ad esempio 32 bit) fa uso di porte con un **fan-in molto elevato**: non praticabile!!
  - Soluzione: **addizionatore veloce a blocchi**

Addizione veloce a blocchi

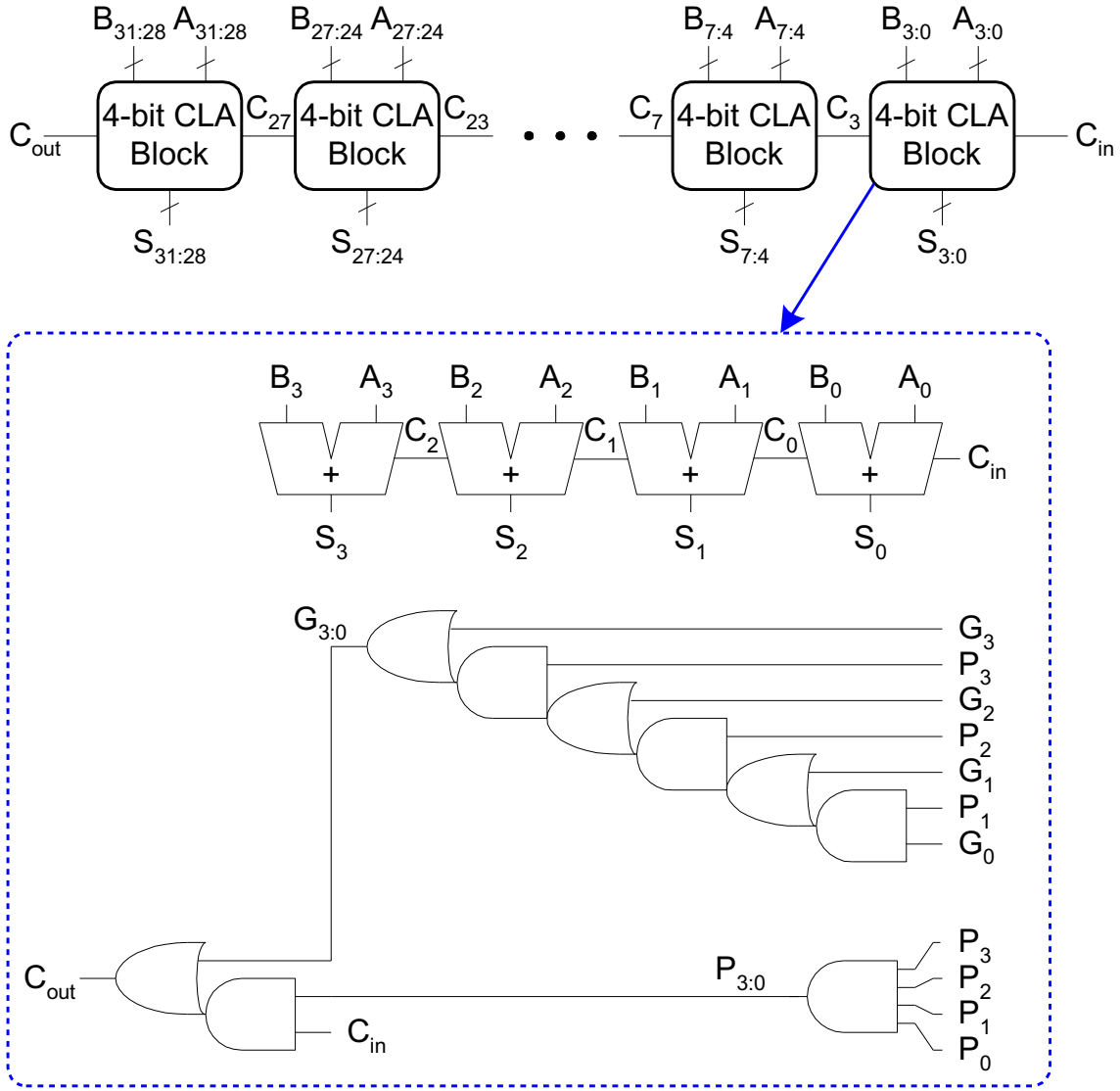
- Il sommatore completo a *n* bit è ottenuto utilizzando un insieme di blocchi costituiti da CLA a *m* bit e della logica CLA
  - La struttura del sommatore a blocchi può essere di tipo *Ripple-Carry* o nuovamente di tipo *Carry-Look-Ahead*
- Esempio: blocco è costituito da un sommatore CLA a 4 bit (ragionevole)

- **Struttura del blocco di un CLA a 4 bit**
  - Il riporto finale di questo sommatore ha la seguente espressione:

    $$c_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0\,c_0$$

  - che può essere riscritta come $c_{uscita} = G + Pc_0$
  - *con* $P = P_3P_2P_1P_0$ e $G = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$
  - Il tempo di ritardo per il calcolo di P e G pari a 2 e 3 porte logiche

Esempio di Sommatore Ripple-Carry con blocchi CLA a 4-bit





POLITECNICO
MILANO 1863

Reti Logiche

Addizionatori per numeri relativi

Addizione e Sottrazione in complemento a 2

- Regole per la **somma** e **sottrazione** di due numeri **in complemento a 2**
  - Per calcolare x+y
    - Fornire in ad un sommatore binario naturale le codifiche in complemento a due degli operandi
    - Ignorare il bit di riporto in uscita
    - Il risultato è in complemento a due
  - Per calcolare x-y
    - Ricavare la rappresentazione dell'opposto di y (complemento a due)
    - Sommare i valori così ottenuti come nella regola precedente x+(-y)
    - Il risultato è in complemento a due
- I risultati sono corretti se e solo se, disponendo di un sommatore binario senza segno ad *n* bit, il risultato sta nell'intervallo:

$$-2^{n-1} \leq x \pm y \leq 2^{n-1}-1$$

- In caso contrario si verifica overflow (o underflow) aritmetico

Addizione e Sottrazione in complemento a 2

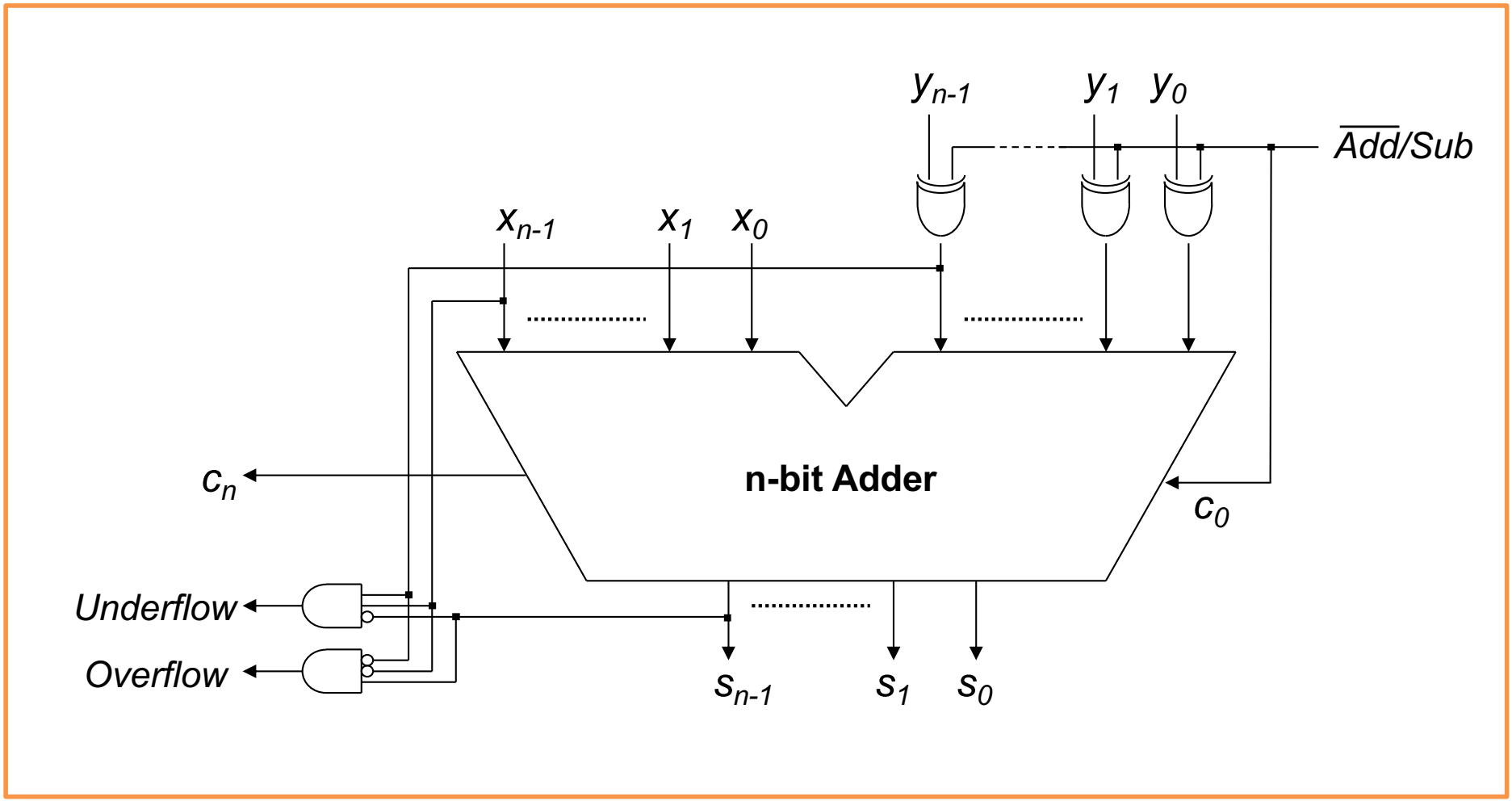
- Condizioni di **overflow** e di **underflow** per somme e sottrazioni in complemento a 2 su *n* bit

A+B			
A	B	Segno somma	Ov/Un
> 0	> 0	1	Si-Ov
> 0	< 0		no
< 0	> 0		no
< 0	< 0	0	Si-Un

A-B=A+(-B)				
A	B	$-B = B_{CPL2}$	Segno somma	Ov/Un
> 0	> 0	< 0		no
> 0	< 0	>0	1	Si-Ov
< 0	> 0	< 0	0	Si-Un
< 0	< 0	> 0		no

- **overflow** per somma = 0 0 1 (segno addendi e segno somma)
- **underflow** per somma = 1 1 0
- **overflow** per sottrazione = 0 1 1
- **underflow** per sottrazione = 1 0 0

Sommatore/Sottrattore in complemento a 2





POLITECNICO
MILANO 1863

Reti Logiche

Moltiplicatori per numeri naturali

Moltiplicazione fra interi senza segno

- La moltiplicazione di numeri **senza segno** si esegue con lo stesso metodo usato per la moltiplicazione decimale
- Il **prodotto** di due numeri binari di *n* e *k* bit è un numero binario di *n+k* bit
- Ogni prodotto parziale deve essere esteso a *n+k* bit tramite 0
- Ad esempio:

```
    1101  ×      ⟶  Moltiplicando M = 13
    1011  =      ⟶  Moltiplicatore Q = 11
-----------
00001101
0001101          Matrice dei
0000000          prodotti parziali
01101
-----------
10001111         ⟶  Prodotto P = 143
```

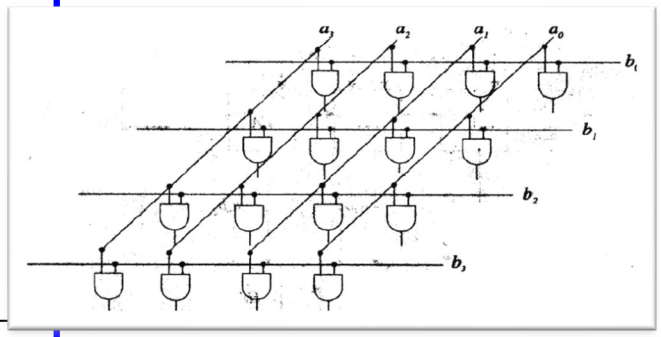
Matrice dei prodotti parziali

- Esempio: Prodotto di due numeri positivi di 4 bit

Moltiplicazione bit a bit

				a_3	a_2	a_1	a_0	×
				b_3	b_2	b_1	b_0	=
				b_0a_3	b_0a_2	b_0a_1	b_0a_0	
			b_1a_3	b_1a_2	b_1a_1	b_1a_0		
		b_2a_3	b_2a_2	b_2a_1	b_2a_0			
	b_3a_3	b_3a_2	b_3a_1	b_3a_0				
p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0	

Matrice di **prodotti parziali** costituita da *n* righe



Architetture di moltiplicatori

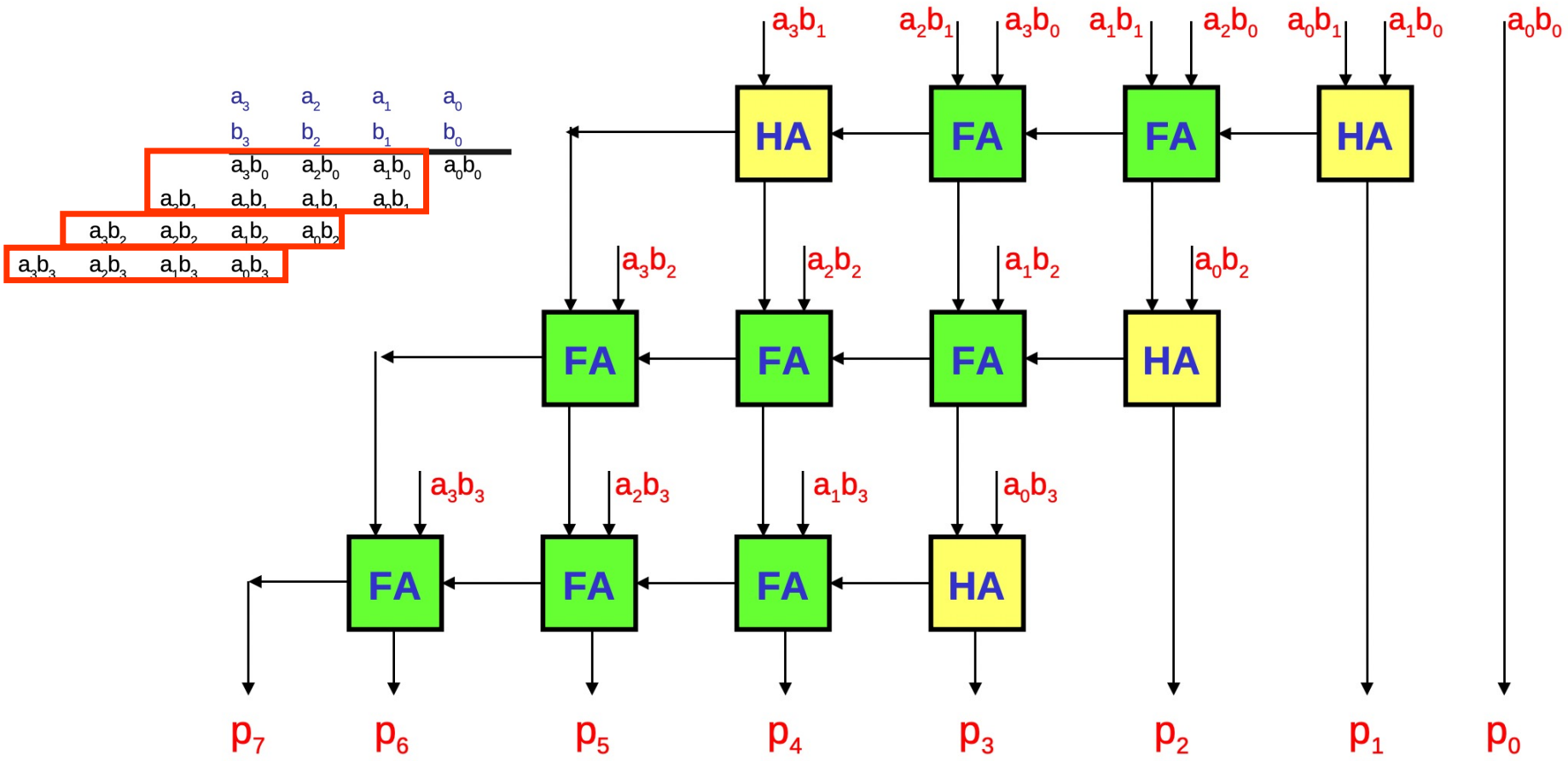
- Le architetture sono definite a seconda del “meccanismo” di somma e propagazione dei riporti delle righe della matrice di prodotti parziali

- Tre architetture principali di moltiplicatori
  - **Somma per righe**: i riporti si propagano per righe
  - **Somma per diagonali**: i riporti si propagano per diagonali
  - **Somma per colonne**: i riporti si propagano per colonne

- Le architetture risultanti sono caratterizzate da strutture regolari

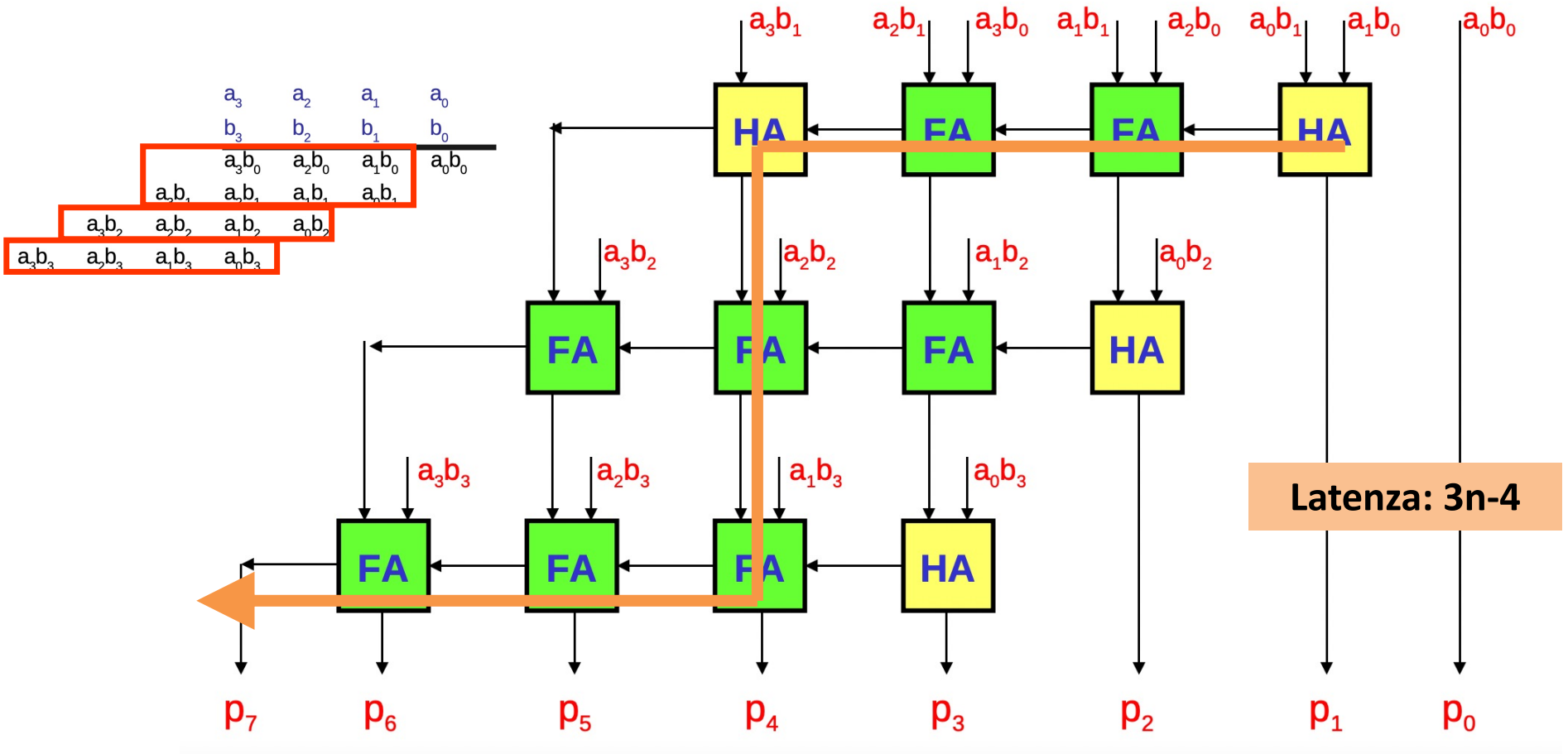
Moltiplicatore con somma per righe

- **n-1** sommatori a **n** bit che lavorano sulle righe della matrice dei prodotti parziali



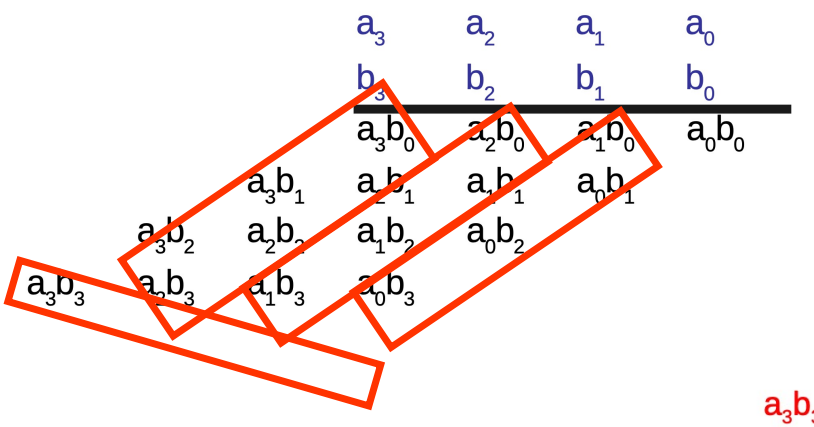
Moltiplicatore con somma per righe

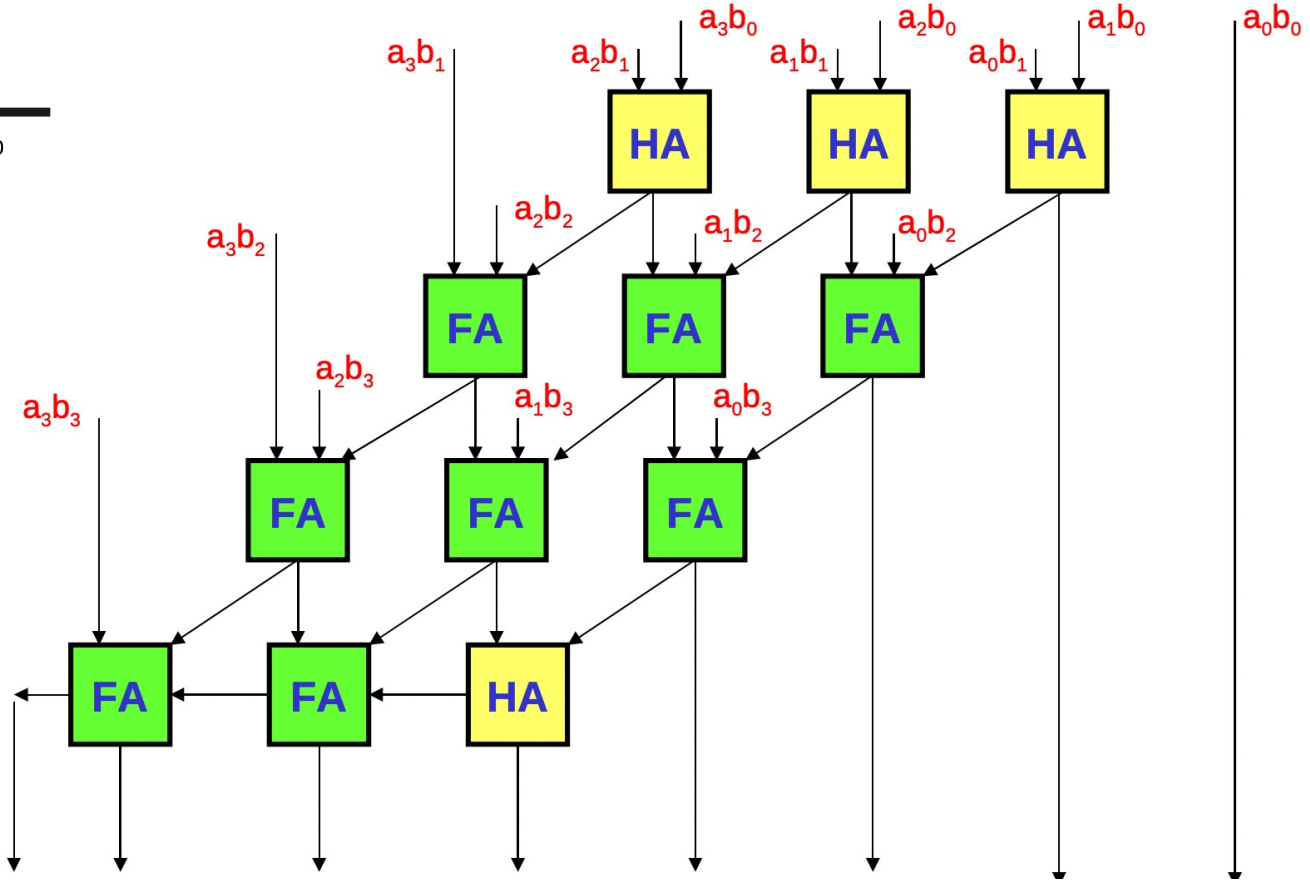
- **n-1** sommatori a **n** bit che lavorano sulle righe della matrice dei prodotti parziali



Moltiplicatore con somma per diagonali

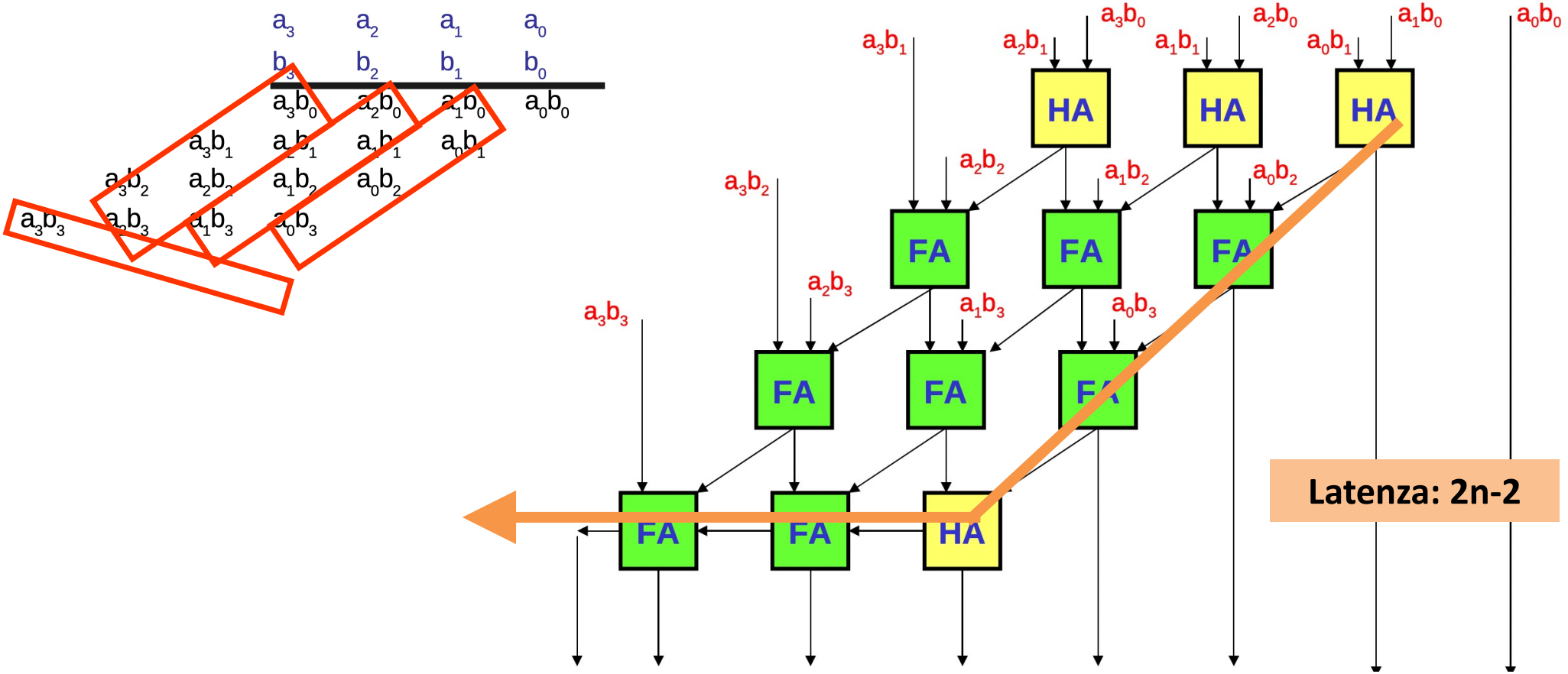
- **n** sommatori a **n-1** bit *in diagonale* sulla matrice dei prodotti parziali





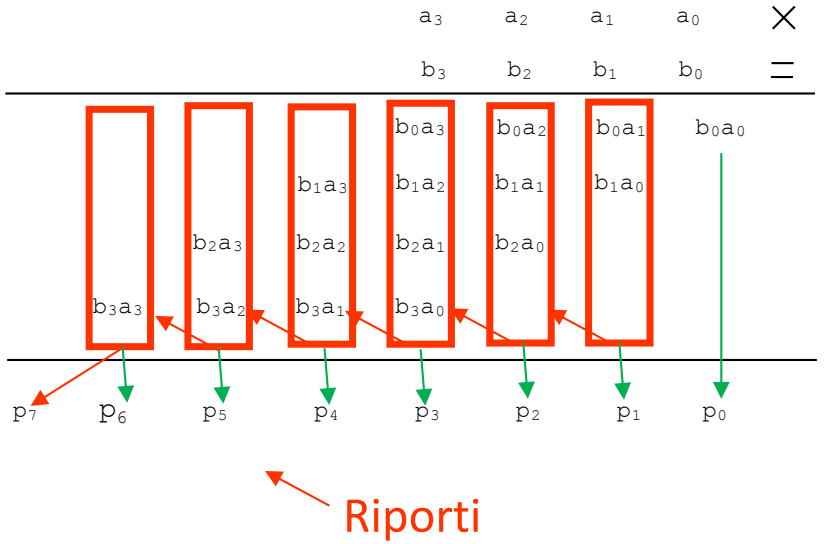
Moltiplicatore con somma per diagonali

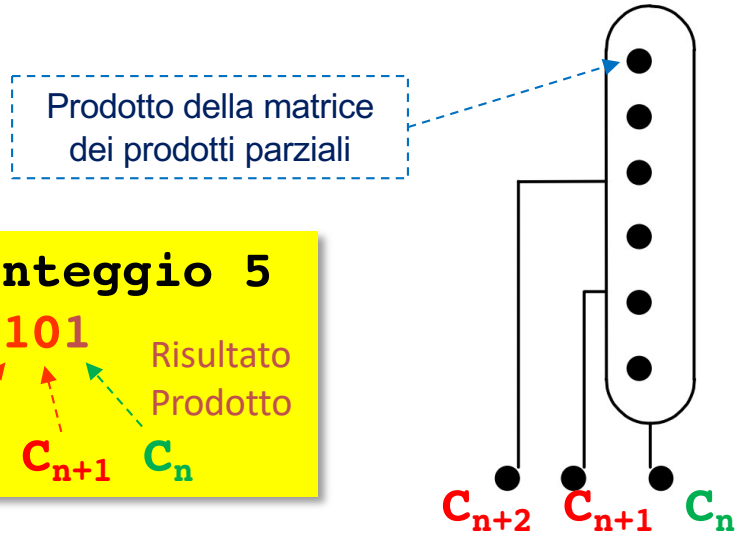
- **n** sommatori a **n-1** bit *in diagonale* sulla matrice dei prodotti parziali



Moltiplicatore con somma per colonne

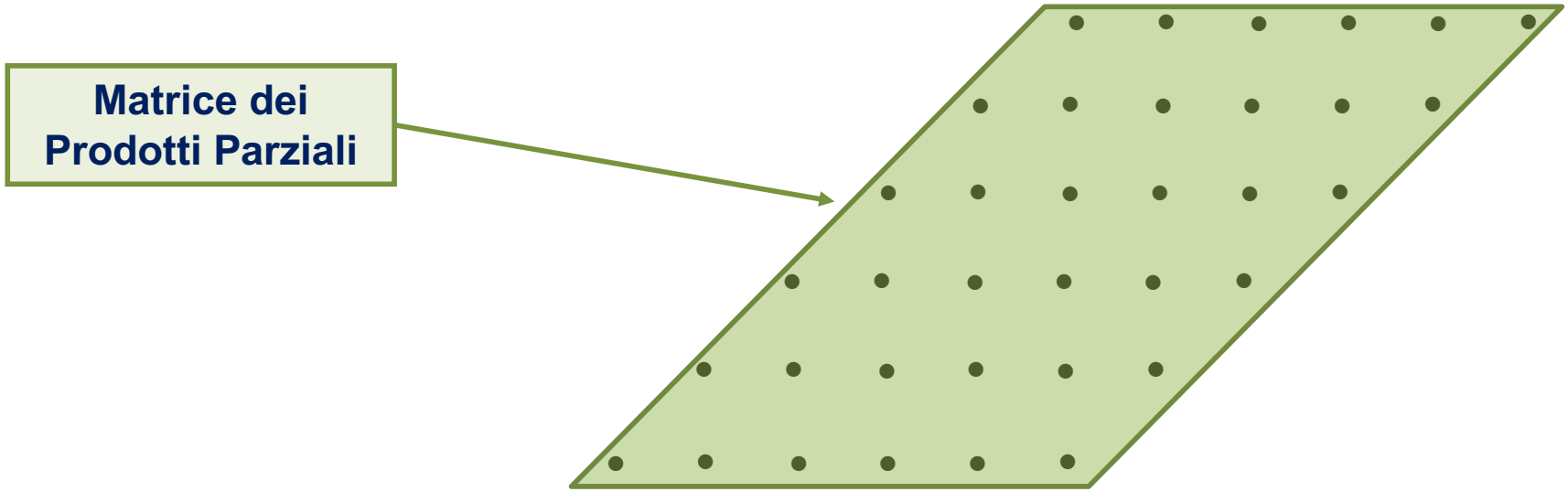
- Il metodo è simile a quello utilizzato a mano per effettuare la moltiplicazione
- Si usano un insieme di **contatori paralleli** che ricevono in ingresso le **colonne dei prodotti parziali** (e gli eventuali riporti dagli stadi precedenti) e generano il **conteggio** degli 1 della colonna
- Il conteggio generato in ogni stadio produce
  - il **bit del prodotto per lo stadio** considerato
  - eventuali **riporti per gli stadi successivi**





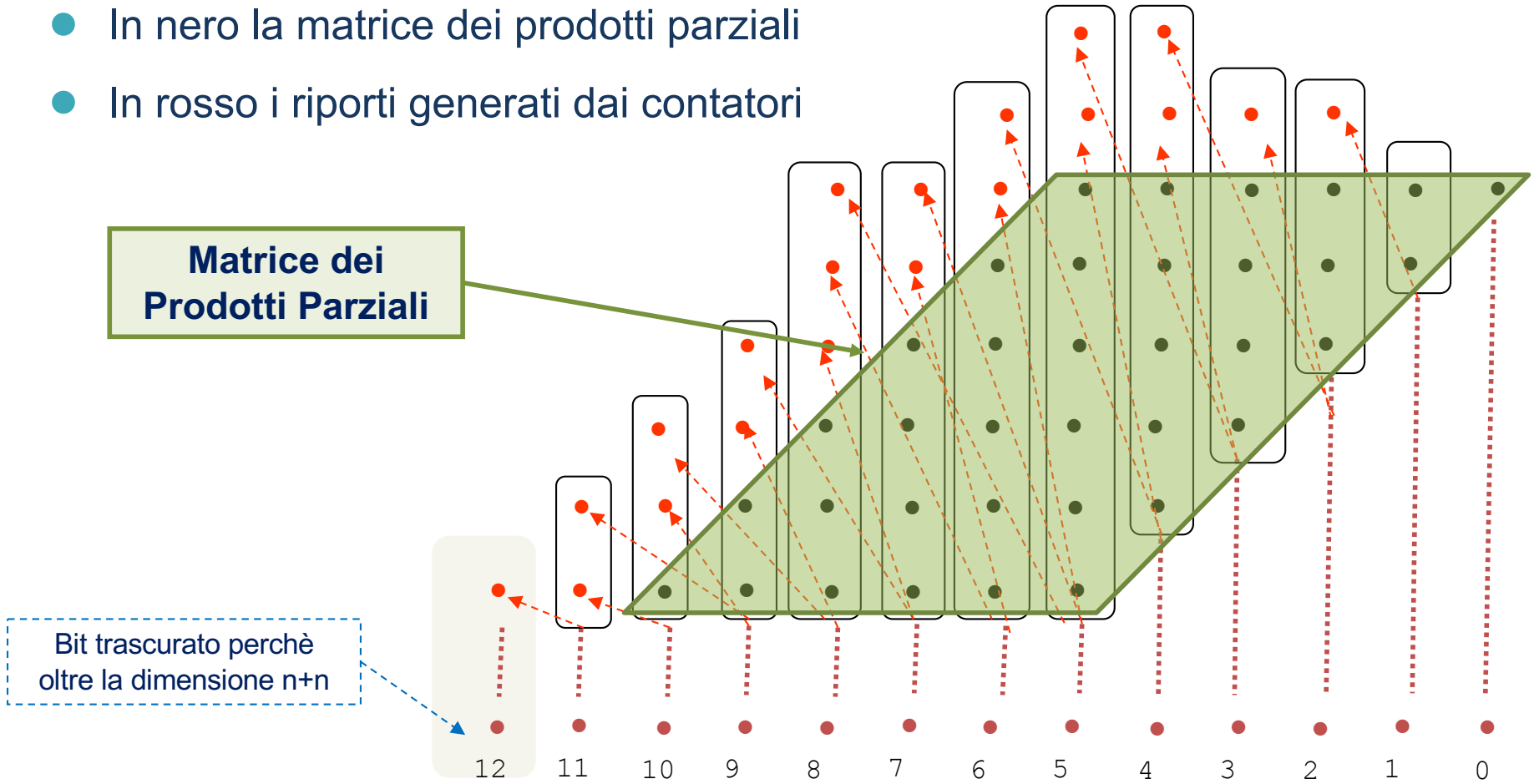
Moltiplicatore con somma per colonne (es.6bit)

- Esempio: Moltiplicando e moltiplicatore da 6 bit
  - In nero la matrice dei prodotti parziali



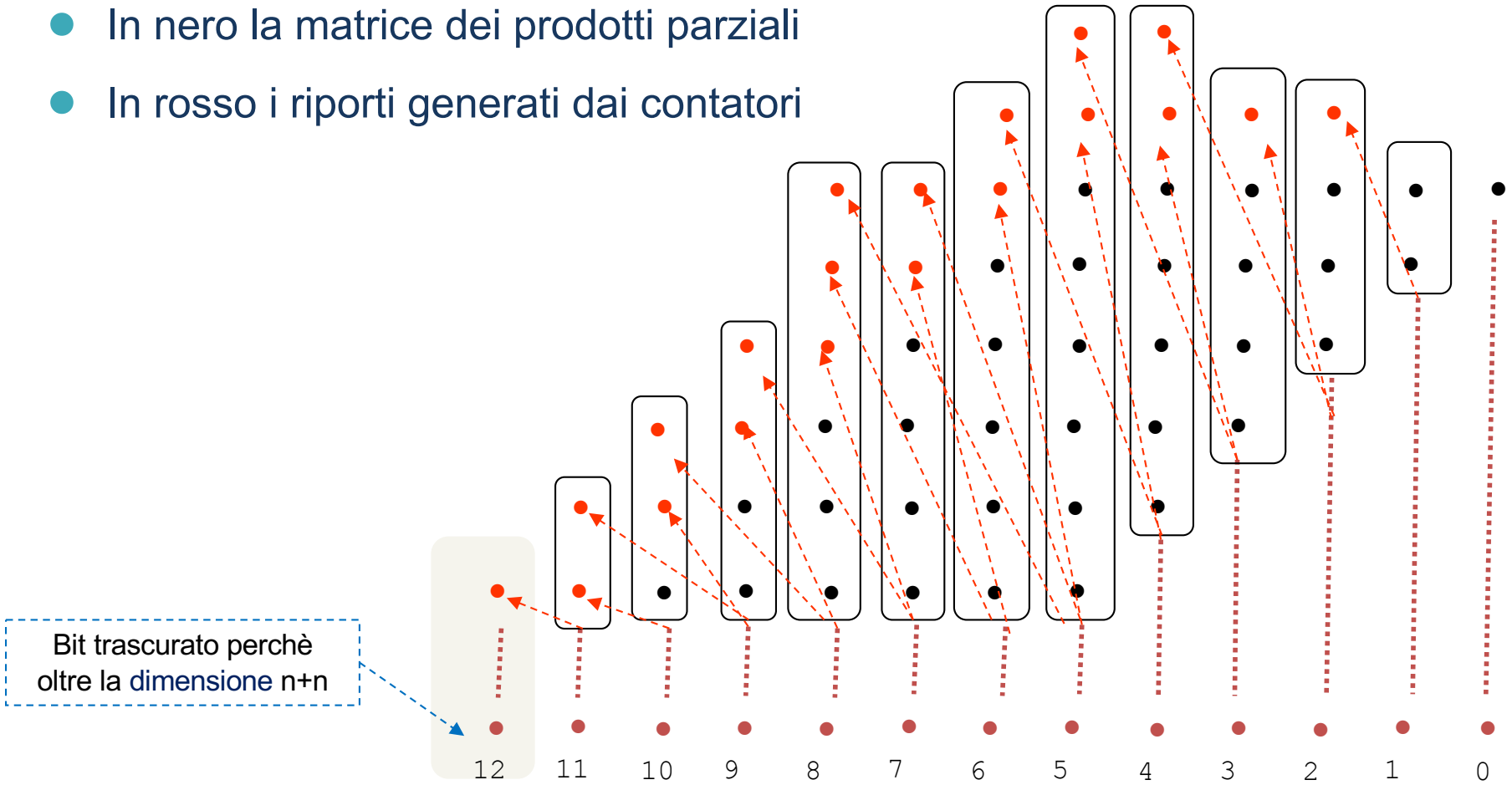
Moltiplicatore con somma per colonne (es.6bit)

- Esempio: Moltiplicando e moltiplicatore da 6 bit
  - In nero la matrice dei prodotti parziali
  - In rosso i riporti generati dai contatori



Moltiplicatore con somma per colonne (es.6bit)

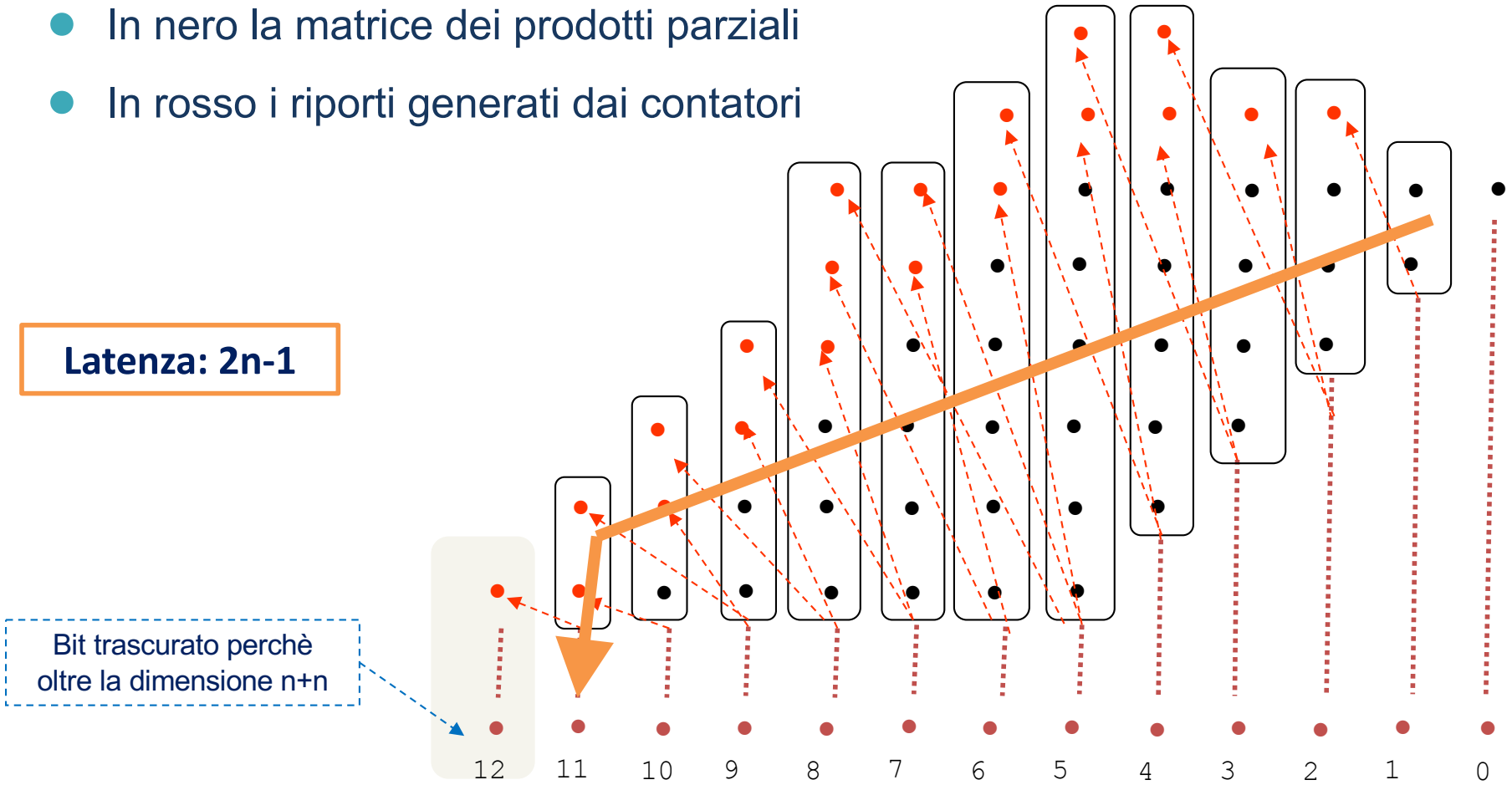
- Esempio: Moltiplicando e moltiplicatore da 6 bit
  - In nero la matrice dei prodotti parziali
  - In rosso i riporti generati dai contatori



Moltiplicatore con somma per colonne (es.6bit)

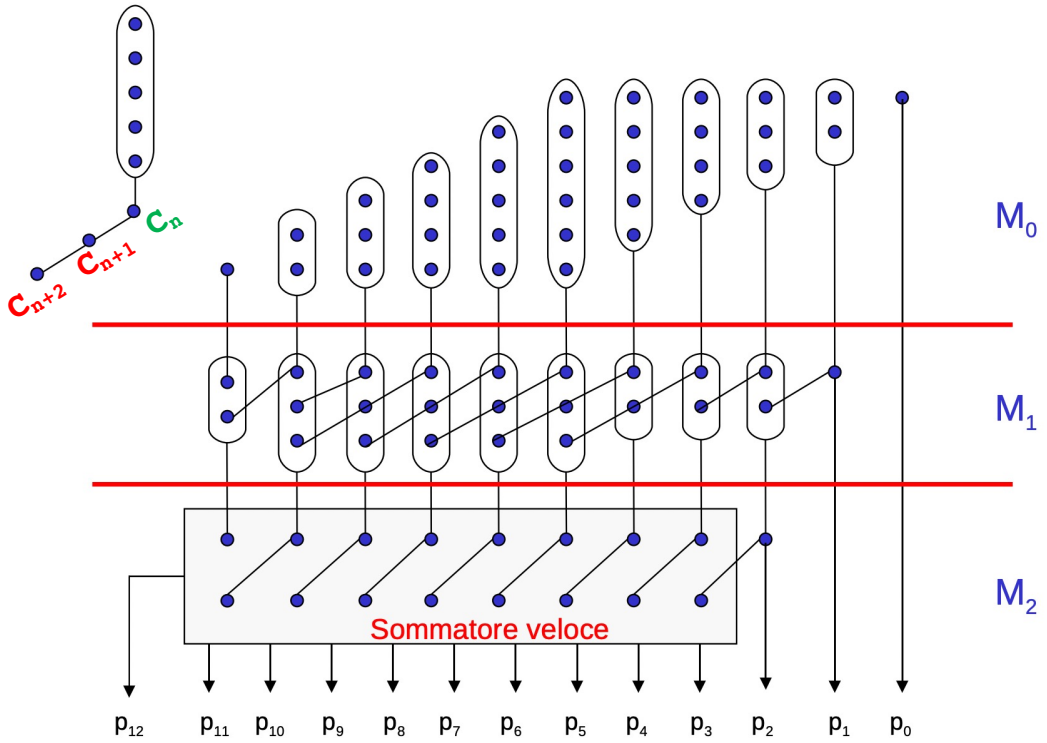
- Esempio: Moltiplicando e moltiplicatore da 6 bit
  - In nero la matrice dei prodotti parziali
  - In rosso i riporti generati dai contatori

Latenza: 2n-1

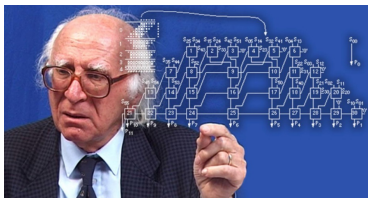


Somma per colonne: Schema Alternativo

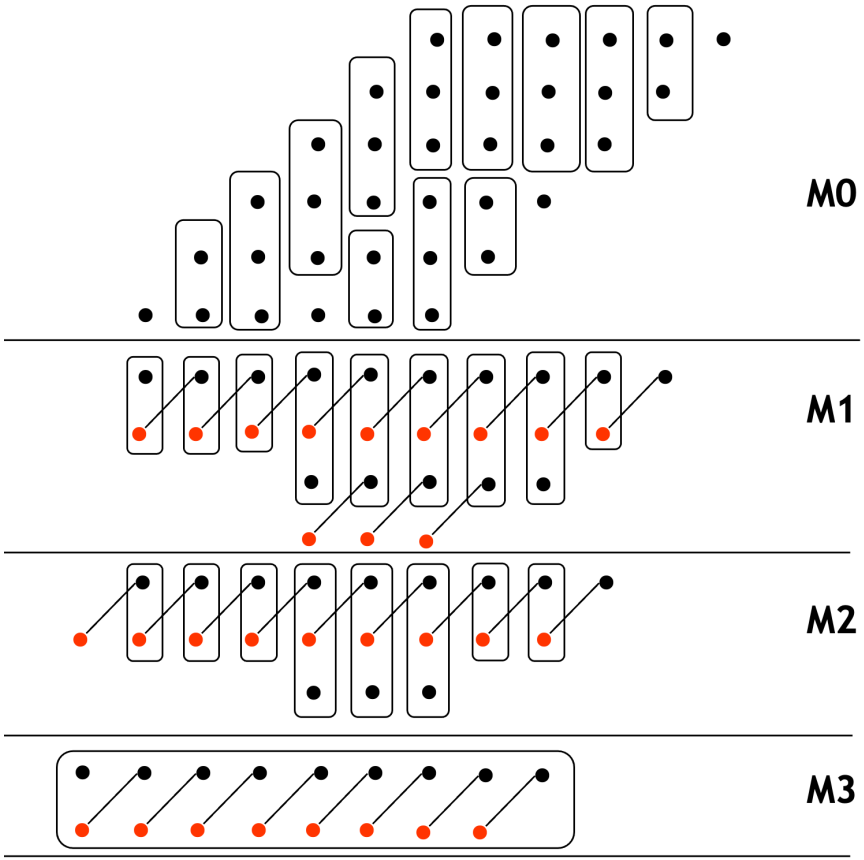
- Idea: evitare la propagazione dei riporti
- **Vantaggi**
  - Velocità (ritardo dominato dal sommatore finale)
  - Un solo addizionatore veloce (e costoso)
- **Svantaggi**
  - Non regolarità
  - contatori con numeri diversi (e potenzialmente alti) di ingressi
- **Soluzioni**
  - Moltiplicatori di Wallace e di Dadda
  - Contatori a soli 2 e 3 bit (FA e HA) e stadi intermedi multipli



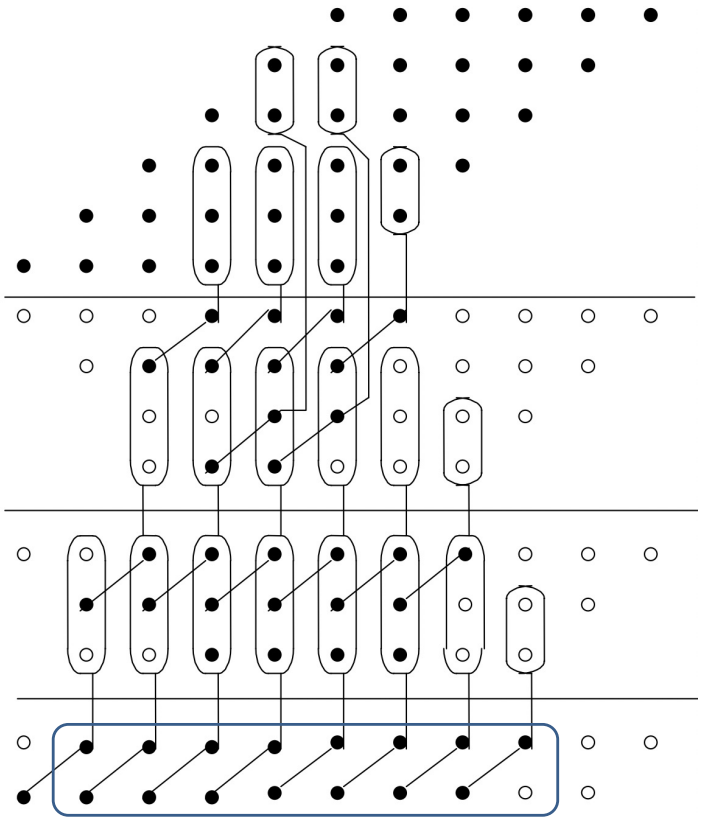
Moltiplicatore di Wallace e di Dadda



WALLACE



DADDA





POLITECNICO
MILANO 1863

Reti Logiche

Moltiplicatori per numeri relativi

Moltiplicazione con segno

- Metodo Naïve
  - Si moltiplicano i valori assoluti degli operandi
  - Si ricava il segno del risultato (e lo si trasforma se serve) in baso al segno degli operandi
  - Algoritmo adoperato nel caso di Modulo e segno ma scomodo in complemento a 2
- Il prodotto nelle moltiplicazioni con segno con numeri in complemento a 2 richiede una **differente gestione della matrice dei prodotti parziali**
- Ricordiamo che nel complemento a 2 il valore della cifra associata al bit più significativo ha segno negativo ($-m_{n-1}2^{n-1}$)
  - Esempio: -5 = 1 0 1 1
    - $-5 = -2^3 + 2^1 + 2^0 = -8 + 2 + 1$

Moltiplicazione con segno (cont.)

- Considerando ad esempio 4 bit, i **valori** di un generico moltiplicando M e di un moltiplicatore Q in complemento a due si ha:
  - $M = (-m_3)2^3+m_2 2^2+m_1 2^1+m_0$ m_3-m_0 sono i bit del moltiplicando
  - $Q = (-q_3)2^3+q_2 2^2+q_1 2^1+q_0$ q_3-q_0 sono i bit del moltiplicatore
- Il segno di M e Q è portato rispettivamente dai bit m_3 e q_3
- La costruzione dei prodotti parziali **tiene conto** del segno dei due fattori (caso M e Q negativi)

				$-q_0m_3$	q_0m_2	q_0m_1	q_0m_0
			$-q_1m_3$	q_1m_2	q_1m_1	q_1m_0	
		$-q_2m_3$	q_2m_2	q_2m_1	q_2m_0		
	q_3m_3	$-q_3m_2$	$-q_3m_1$	$-q_3m_0$			

Moltiplicazione con segno (cont.)

- Scomposizione della matrice iniziale in due **sottomatrici**

			0	q_0m_2	q_0m_1	q_0m_0
		0	q_1m_2	q_1m_1	q_1m_0	
	0	q_2m_2	q_2m_1	q_2m_0		
q_3m_3	0	0	0			
Somma parziale termini positivi						-
			q_0m_3	0	0	0
		q_1m_3	0	0	0	
	q_2m_3	0	0	0		
0	q_3m_2	q_3m_1	q_3m_0			
Somma parziale termini negativi						

M+

M−

- Il risultato è dato dalla differenza dei due risultati parziali: ***M+ - M-***

Prodotto tra numeri relativi - Esempio

➡ A = 00101, B = 10101 0 : perché appartenenti all'altra matrice

```
      00101
      10101
-------------
      00101
     00000
    00101
   00000
  00101
```

```
      00101
      10101
-------------
      00101
     00000
    00101
   00000
  00000
-------------
P=  000011001
```

Matrice contributi positivi

```
      00101
      10101
-------------
      00000
     00000
    00000
   00000
  00101
-------------
N=  001010000
```

Matrice contributi negativi

Risultato = P – N = P + (-N) :

```
(25)   0000011001 +
(-80)  1110110000
-----------------
       1111001001
```

Moltiplicazione con segno - Algoritmo di Booth

- Un'altra tecnica usata per effettuare la moltiplicazione è quella basata sull'algoritmo di Booth che usa al suo interno sommatori/sottrattori in complemento a due
- Si consideri ad esempio la moltiplicazione per Q=30:

$$M \times 30 = M \times (32 - 2) = M \times 32 - M \times 2$$

- In rappresentazione binaria:

$$M \times 0011110 = M \times 0100000 - M \times 0000010$$
$$= M \times 0100000 + M_{(-)} \times 0000010$$

- I moltiplicatori così ottenuti
  - Sono potenze del due
  - Sono sequenze di bit con un solo uno

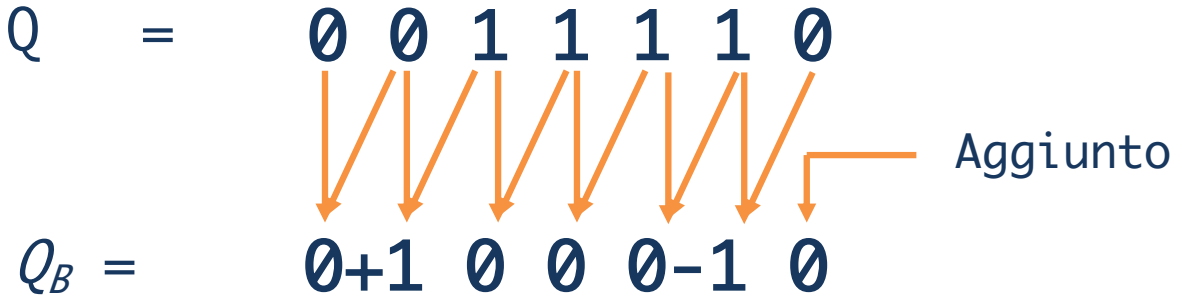
Algoritmo di Booth: codifica del moltiplicatore

- L'algoritmo si basa sulla scomposizione appena vista
- Tale scomposizione è rappresentata come una **codifica del moltiplicatore** basata sulle seguenti regole
  - Si scorre il moltiplicatore da sinistra verso destra
  - Il **moltiplicatore codificato Q_B** si ottiene come segue:
    - Uso il simbolo 0 quando due bit successivi sono uguali
    - Uso il simbolo +1 quando si passa da 0 ad 1
    - Uso il simbolo -1 quando si passa da 1 a 0
    - Infine
      - Se Q termina con 0 aggiungo 0 a Q_B
      - altrimenti aggiungo -1
      - *… come se esistesse un «ghost zero» alla fine del numero*

Moltiplicatore		Codifica	PP_i
q_i	q_{i-1}		
0	0	0	$0 \times M = 0$
0	1	+1	$+1 \times M = M$
1	0	-1	$-1 \times M = M_{(-)}$
1	1	0	$0 \times M = 0$

Algoritmo di Booth: esempio di codifica

- Ad esempio $Q = 30$ appena visto su 7 bit è codificato come segue:



- Utilizzando tale codifica, i prodotti parziali saranno:
  - *0* con **estensione del segno**, quando $q_{B,i} = 0$
  - *M* con **estensione del segno**, quando $q_{B,i} = +1$
  - $M_{(-)}$ con **estensione del segno**, quando $q_{B,i} = -1$

Esempio

- Moltiplicare 13 x -9, usando l'algoritmo di Booth su 5 bit
- I valori binari da usare sono:
  - 13 = 01101 -13 = 10011
  - -9 = 10111 -9_{Booth} = -1+100-1
- Il prodotto si esegue quindi nel modo seguente:

Estensione a n+k bit necessaria tramite estensione del segno

```
                    0 1 1 0 1 ×        → Moltiplicando M = 13
                   -1+1 0 0-1 =        → Moltiplicatore Q = -9
                  ---------------------
M(-)    1 1 1 1 1 1 0 0 1 1
0       0 0 0 0 0 0 0 0 0 0
0       0 0 0 0 0 0 0 0 0
M       0 0 0 1 1 0 1
M(-)    1 1 0 0 1 1
                  ---------------------
        1 1 1 0 0 0 1 0 1 1        → Prodotto P = -117
```

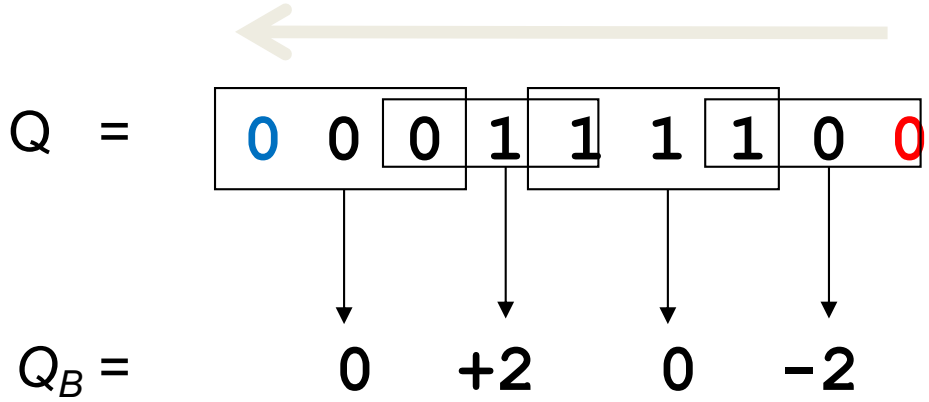
Booth Radix-4 – Codifica del moltiplicatore

- L'algoritmo di Booth appena visto viene detto Radix-2
- Il numero dei prodotti parziali nel Radix-2 sono gli stessi
- Una riduzione può avvenire per radici di ordine superiore
  - Esempio: Radix-4 dove vengono usati 3 bit del moltiplicatore per determinare la sua codifica

Moltiplicatore			$-2q_{i+1}+q_i+q_{i-1}$
q_{i+1}	q_i	q_{i-1}	Codifica
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	2
1	0	0	-2
1	0	1	-1
1	1	0	-1
1	1	1	0

Booth Radix 4 - Esempio di codifica

- Esempio: Consideriamo un moltiplicatore Q = 30
  - La codifica parte da destra verso sinistra saltando un bit
    - Il primo gruppo include solo 2 bit (zero aggiunto a destra)
  - Si codificano i bit in posizione «dispari»



  - Nota: il numero dei prodotti parziali in radix-2 è di 7 mentre in radix-4 è di soli 4
  - ***ATTENZIONE!! Per la codifica si parte sempre da 1 bit in più a DX (pari a zero) e si estende il segno fino a completare i bit necessari per la codifica***

Esempio

- Moltiplicare 13 x -9, usando l'algoritmo di Booth Radix-4 su 5 bit:
  - *13* = 01101
  - -9=10111 → -9 =1101110 → gruppi di -9_B=110;011;110 = -1; +2; -1
- Il prodotto si esegue quindi nel modo seguente:
  - Nota: ogni prodotto parziale, in radix-4, è traslato a sinistra di due posizioni

```
                0 1 1 0 1 ×      → Moltiplicando M = 13
                  -1+2-1 =       → Moltiplicatore Q = -9
-------------------------
1 1 1 1 1 1 0 0 1 1
0 0 0 1 1 0 1 0 - -
1 1 0 0 1 1 - - - -
-------------------------
1 1 1 0 0 0 1 0 1 1              → Prodotto P = -117
```



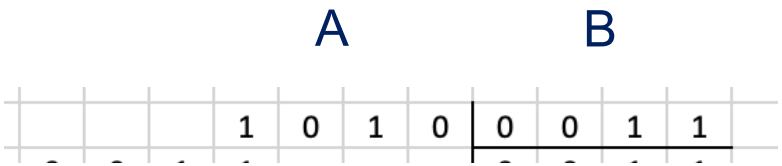
POLITECNICO
MILANO 1863

Reti Logiche

Divisore per numeri naturali

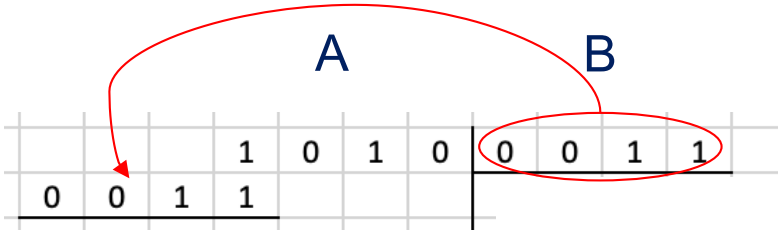
Divisione tra numeri binari

- Esempio: si vuole fare la divisione a 4 bit tra 1010 e 0011
  - 1010 dividendo
  - 0011 divisore



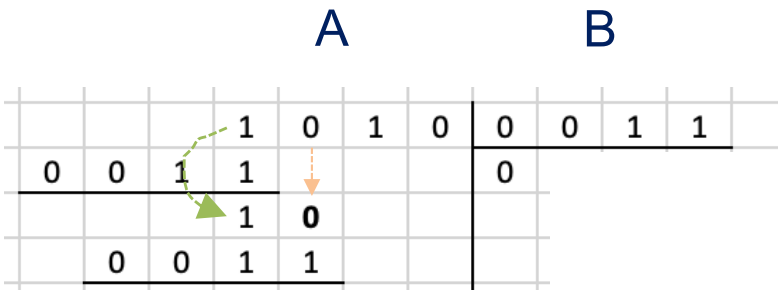
Divisione tra numeri binari

- Esempio: si vuole fare la divisione a 4 bit tra 1010 e 0011
  - 1010 dividendo
  - 0011 divisore



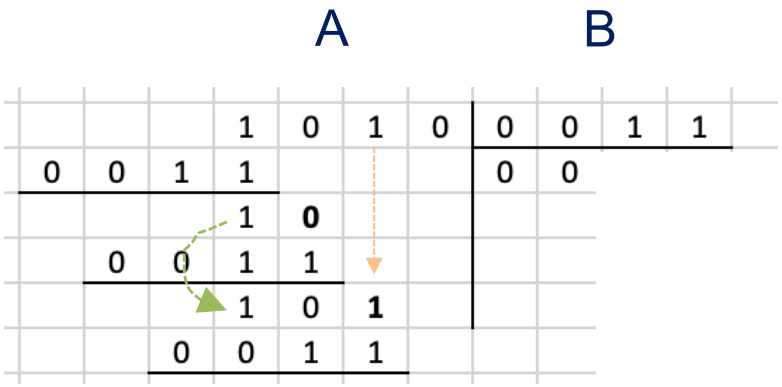
Divisione tra numeri binari

- Esempio: si vuole fare la divisione a 4 bit tra A=1010 e B=0011
  - 1010 dividendo A
  - 0011 divisore B



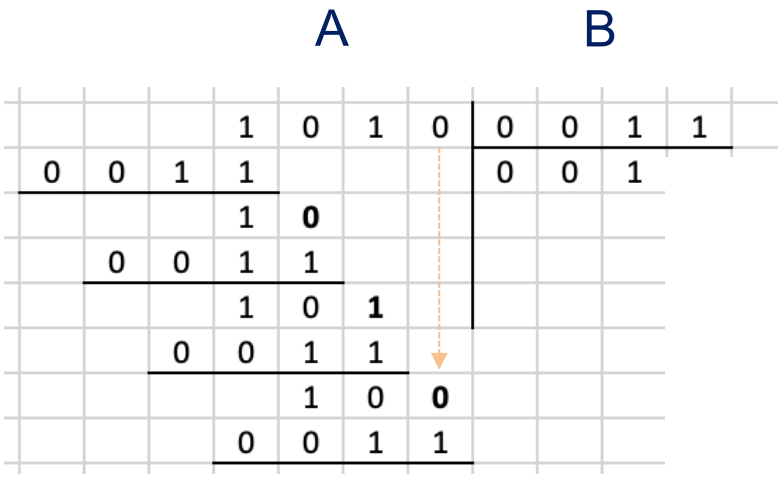
Divisione tra numeri binari

- Esempio: si vuole fare la divisione a 4 bit tra A=1010 e B=0011
  - 1010 dividendo A
  - 0011 divisore B



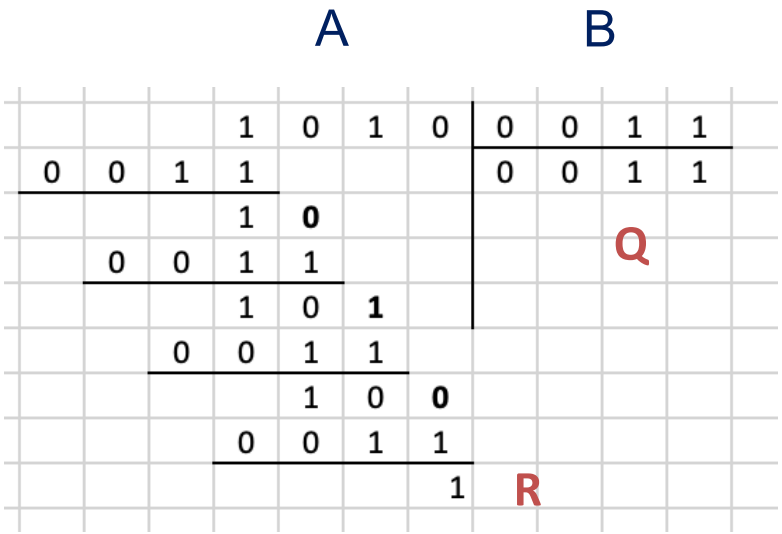
Divisione tra numeri binari

- Esempio: si vuole fare la divisione a 4 bit tra A=1010 e B=0011
  - 1010 dividendo A
  - 0011 divisore B

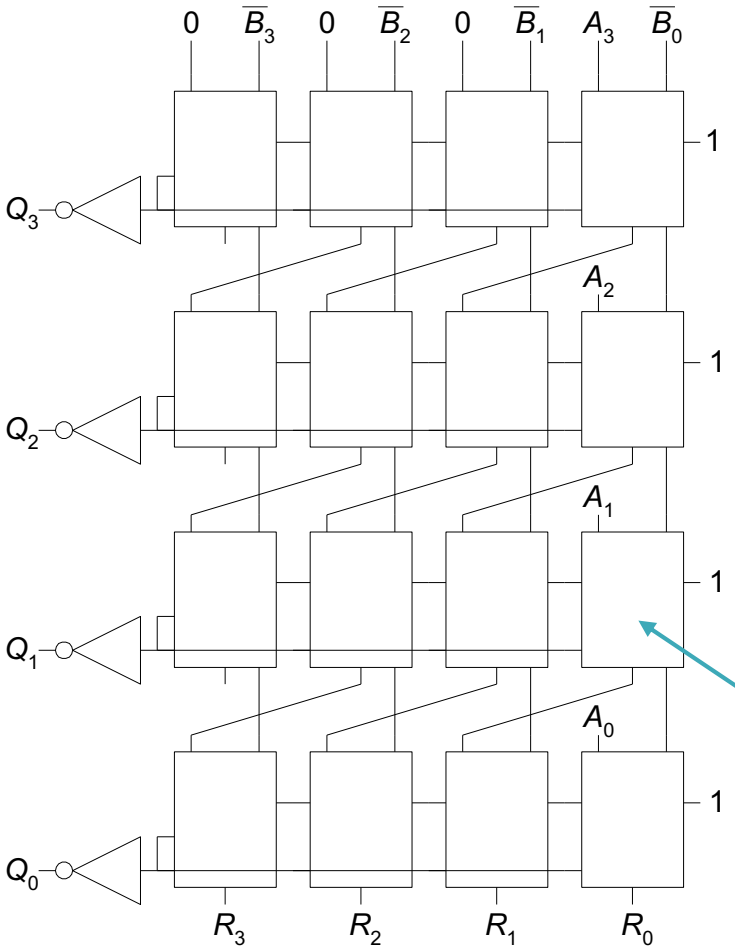


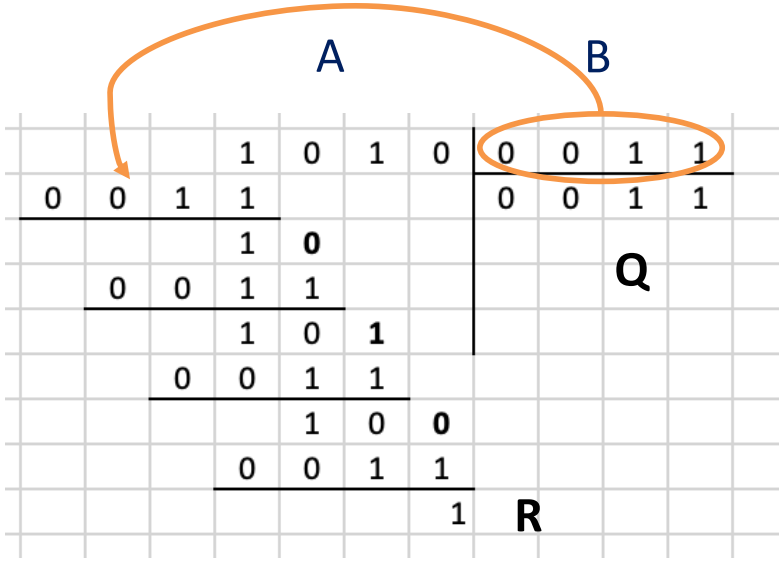
Divisione tra numeri binari

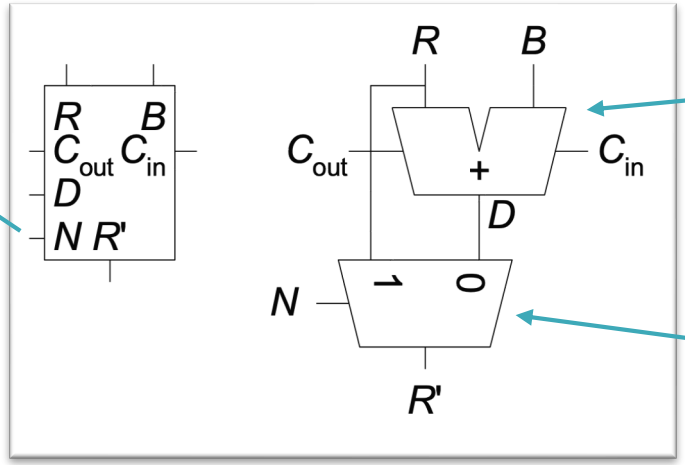
- Esempio: si vuole fare la divisione a 4 bit tra 1010 e 0011
  - 1010 dividendo
  - 0011 divisore



Divisore (esempio 4x4)







Effettua la sottrazione tra il dividendo parziale e il divisore

Effettua la selezione tra il dividendo parziale e il risultato della sottrazione



POLITECNICO
MILANO 1863

Reti Logiche

Rappresentazione dei Numeri in Virgola Fissa

Numeri in virgola fissa

- Fino a questo punto abbiamo assunto che
  - Un vettore di bit rappresentasse sempre un numero intero
  - Eventualmente con segno
- Tutte le considerazioni fatte fino ad ora e tutti i metodi esposti continuano a valere se si attribuisce ai vettori di bit il significato di numeri in *virgola fissa*
- Un sistema di numerazione in virgola fissa è quello in cui:
  - La posizione della virgola decimale è implicita
  - La posizione della virgola decimale uguale in tutti i numeri

Numeri in virgola fissa: fattore di scala

- La posizione della virgola equivale alla interpretazione del valore intero moltiplicato per un fattore di scala
- Si consideri ad esempio il vettore di $k+n$ bit (k bit per rappresentare la parte intera e n bit per rappresentare la parte frazionaria):
- Detti V_I il valore intero e V_{VF} il valore in virgola fissa di B:

$$V_{VF}(B) = V_I(B) \times S_n = V_I(B) \times 2^{-n}$$

$$B = b_{k-1} \ldots b_0, b_{-1} \ldots B_{-n}$$

- Il suo valore è dato da

$$V(B) = b_{k-1}*2^{k-1} + \ldots + b_0*2^0 + \underbrace{b_{-1}*2^{-1} + \ldots + b_{-n}*2^{-n}}_{\text{parte frazionaria}}$$

Esempio

- Si vuole derivare il valore del vettore binario rappresentante un numero codificato in virgola fissa facente uso di 5 bit per la parte frazionaria:

  B = 01010110 (010.10110)

- Il valore di *B*, considerandolo intero è:

$$V_I(B) = 2^6 + 2^4 + 2^2 + 2^1 = 64 + 16 + 4 + 2 = 86$$

- Il fattore di scala dovuto ai 5 bit di parte frazionaria è:

$$S_5 = 2^{-5} = 1/32 = 0.03125$$

- Da cui, moltiplicando per il fattore di scala, si ha:

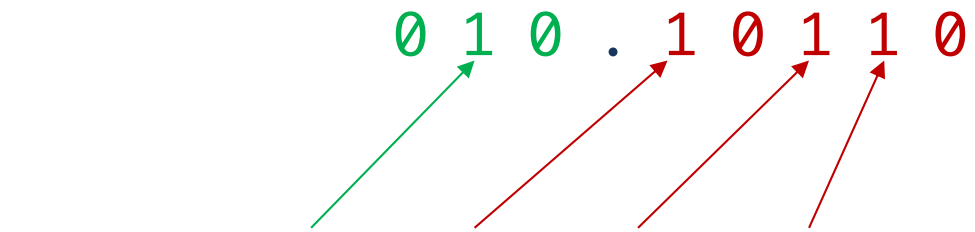
$$V_{VF}(B) = V_I(B) \times S_5 = 86 \times 0.03125 = 2.6875$$

Esempio (cont.)

- Si vuole derivare il valore del vettore binario rappresentante un numero codificato in virgola fissa facente uso di 5 bit per la parte frazionaria:

  B = 01010110 (010.10110)

- Il suo valore in virgola fissa su può anche derivare direttamente ed è:



$$V_{VF}(B) = 2^{1} + 2^{-1} + 2^{-3} + 2^{-4} =$$

$$= 2 + {}^{1}/_{2} + {}^{1}/_{8} + {}^{1}/_{16}$$

$$= {}^{43}/_{16} = 2.6875$$

Operazioni

- Tutte le operazioni aritmetiche con numeri rappresentati in virgola fissa saranno eseguite come se fossero numeri interi
- I circuiti aritmetici sono gli stessi
- Attenzione andrà posta alla posizione della virgola
  - Esempi:
    - Somma e sottrazione:
      - Necessario allineamento delle parti frazionarie
    - Moltiplicazione
      - Posizione della virgola nel risultato è pari alla somma del numero dei bit delle due parti frazionarie



POLITECNICO
MILANO 1863

Reti Logiche

Rappresentazione dei Numeri in Virgola Mobile

Virgola fissa vs virgola mobile

- Nella rappresentazione in **virgola mobile (floating point)** la posizione della virgola è mobile ed è indicata dal valore di un fattore moltiplicativo

$$(\pm \text{Mantissa} \times \text{Base}^{\textit{Esponente}})$$

- A pari numero di bit disponibili
  - con la rappresentazione **intera** o in **virgola fissa**, i valori rappresentati sono **distribuiti uniformemente** nel campo di rappresentabilità
  - con la rappresentazione in **virgola mobile**, i valori rappresentati sono distribuiti **non uniformemente** nel campo di rappresentabilità
    - sono “più fitti” vicino allo 0 e “più radi” per valori assoluti grandi

Rappresentazione in virgola mobile

- La rappresentazione in virgola mobile ha come principale problema quello di non essere una **rappresentazione biunivoca**
  - Allo stesso numero corrispondono più coppie mantissa esponente
  - Esempio: $+ 1.001 \times 2^{12} = + 10.01 \times 2^{11} = + 0.1001 \times 2^{13}$
- Richiede la suddivisione dei bit a disposizione sui campi relative alla mantissa e all'esponente

- Lo **standard IEEE754** definisce un metodo per la rappresentazione dei numeri in virgola mobile
  - Singola precisione 32 bit
  - Doppia precisione 64 bit

La rappresentazione della mantissa

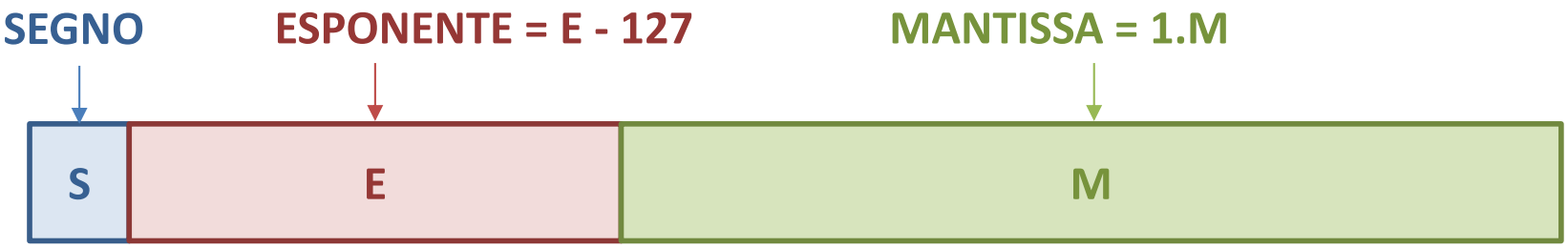
- La mantissa è rappresentata in **modulo e segno**
  - Un bit della codifica verrà usato per rappresentare il segno
- La biunivocità della rappresentazione del numero viene ottenuta tramite la **forma normalizzata**:
  - La prima cifra della mantissa è **sempre 1**
  - Il **primo uno non viene rappresentato** esplicitamente
    - Mantissa = `1.M` dove `M` è il valore rappresentato
  - $1 \leq MANTISSA < 2$
  - Impossibilità di rappresentare il valore 0 che viene gestito con una codifica ad hoc

La rappresentazione dell'esponente in singola precisione

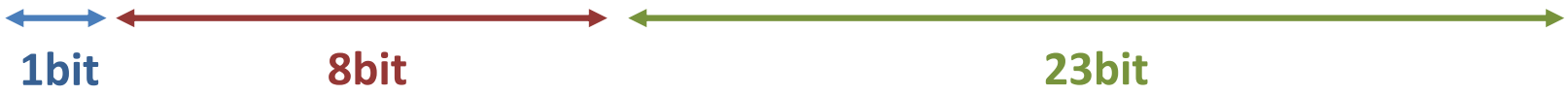
- L'**esponente** utilizza la **codifica in eccesso 127**
  - il valore effettivo dell'esponente è pari a `ESPONENTE=E-127`
  - In doppia precisione l'esponente è codificato in eccesso 1023
- Se 0<E<255
  - $(-1)^S \times 1.M \times 2^{E-127}$
- Altrimenti esistono codifiche particolari:
  - E = 0 e M = 0 rappresenta lo zero (pos/neg)
  - E = 0 e M !=0 rappresenta i numeri non normalizzati
    - $(-1)^S \times 0.M \times 2^{-126}$
  - E = 255 e M = 0 rappresenta infinito (pos/neg)
  - E = 255 e M !=0 rappresenta il NotANumber

Formato in singola e doppia precisione

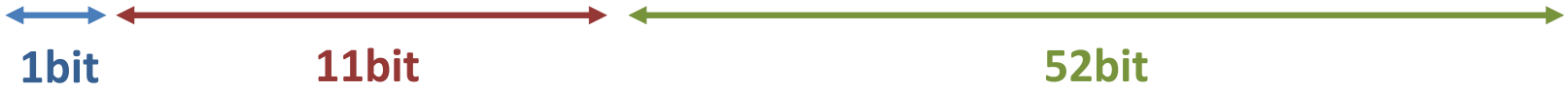
- $\pm$ MANTISSA $\times$ $2^{\text{ESPONENTE}}$ diviene $(-1)^S \times 1.M \times 2^{E-127}$



- Singola precisione (32bit)



- Doppia precisione (64bit)





POLITECNICO
MILANO 1863

Reti Logiche

Operazioni in virgola mobile

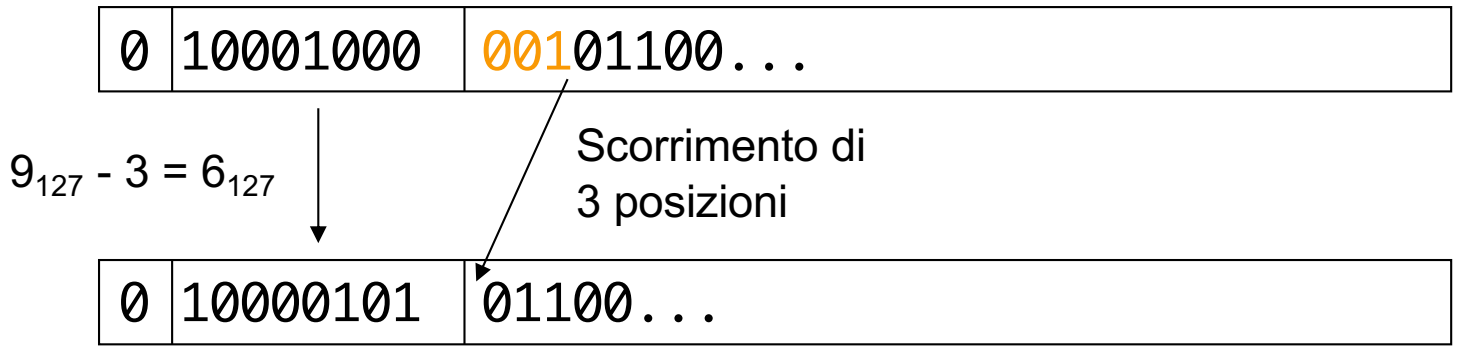
Operazioni in virgola mobile

- Somma/Sottrazione
- Prodotto
- Divisione

- Normalizzazione
- Troncamento

Normalizzazione

- Tutte le operazioni operano su numeri normalizzati
- Se **mancasse l'1 implicito**, la normalizzazione di un numero con mantissa 0+*M'* ed esponente *n*, si esegue come segue:
  - Si fa scorrere verso sinistra la mantissa *M'* fino al primo uno, compreso
  - Sia *k* il numero di posizioni di tale scorrimento, si sottrae *k* all'esponente *n*
- Ad esempio:



- Simile è il caso in cui avessi un **uno in posizione superiore alla prima nella parte intera della mantissa**

Troncamento

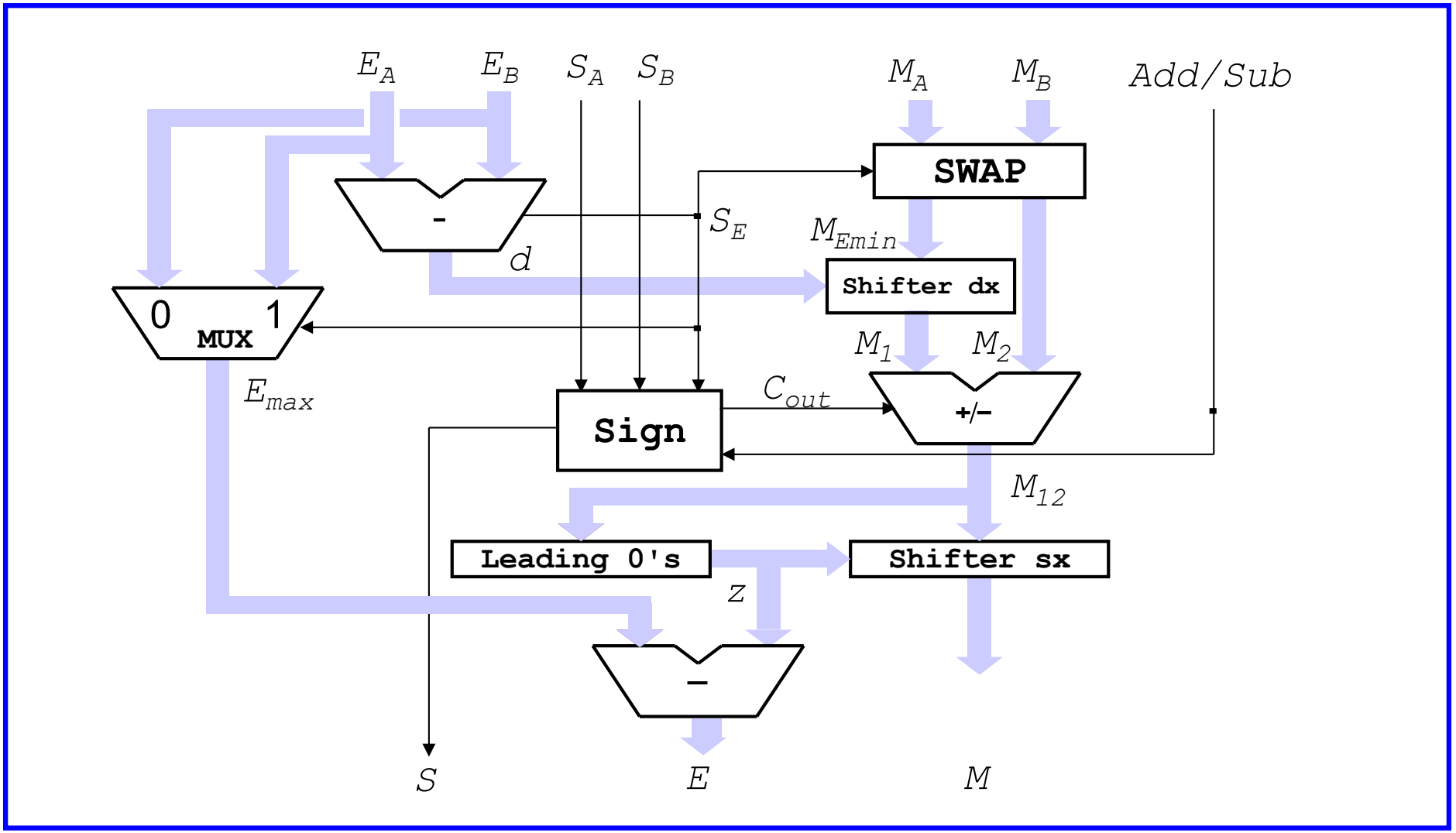
- Il troncamento serve a rimuovere un certo numero di bit per ottenere una rappresentazione approssimata del risultato
- Spesso accade di rappresentare i risultati intermedi di una operazione con una precisione maggiore di quella degli operandi e del risultato

- Il troncamento semplice è solo uno dei metodi possibili definiti nello standard IEEE754
  - Altri metodi di rounding sono presenti nello standard
  - Lo standard non definisce quale metodo

Somma e sottrazione in virgola mobile

- La **somma o sottrazione** tra numeri in virgola mobile viene eseguita secondo i seguenti passi:
  - Si sceglie il numero con esponente minore
  - Si fa scorrere la sua mantissa a destra un numero di bit pari alla differenza dei due esponenti
  - Si assegna all'esponente del risultato il maggiore tra gli esponenti degli operandi
  - Si esegue l'operazione di somma (algebrica) tra le mantisse per determinare il valore ed il segno del risultato
  - Si normalizza il risultato così ottenuto
    - Non sempre quest'ultima operazione è necessaria
    - **Attenzione!!!** Il riporto si può propagare anche dopo la posizione della virgola

Architettura del sommatore in virgola mobile



Moltiplicazione in virgola mobile

- La **moltiplicazione** tra numeri in virgola mobile viene eseguita secondo i seguenti passi:
  - Si sommano gli esponenti e si sottrae 127
    - La sottrazione di 127 dalla somma degli esponenti è necessaria in quanto sono rappresentati in eccesso 127

      $E_{a,127} = E_a + 127$ $\qquad$ $E_{b,127} = E_b + 127$

      $E_{axb,127} = E_{axb} + 127 = (E_a + 127) + (E_b + 127) - 127$
  - Si calcola il risultato della moltiplicazione delle mantisse
  - Si determina il segno del risultato
  - Si normalizza il risultato così ottenuto
    - Non sempre quest'ultima operazione è necessaria

Divisione in virgola mobile

- La **divisione** tra numeri in virgola mobile viene eseguita secondo i seguenti passi:
  - Si sottraggono gli esponenti e si somma 127
    - La somma di 127 alla differenza degli esponenti è necessaria in quanto sono rappresentati in eccesso 127

$$E_{a,127} = E_a + 127 \qquad E_{b,127} = E_b + 127$$

$$E_{a/b,127} = E_{a/b} + 127 = (E_a + 127) - (E_b + 127) + 127$$

  - Si calcola il risultato della divisione delle mantisse
  - Si determina il segno del risultato
  - Si normalizza il risultato così ottenuto
    - Non sempre quest'ultima operazione è necessaria